

Strukture podataka i algoritmi

Algoritmi grafova

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

- 1) Minimalno povezujuće stablo
(Primov algoritam)
- 2) Najkraći putevi u težinskim grafovima
(Dijkstra algoritam)

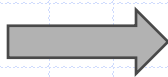
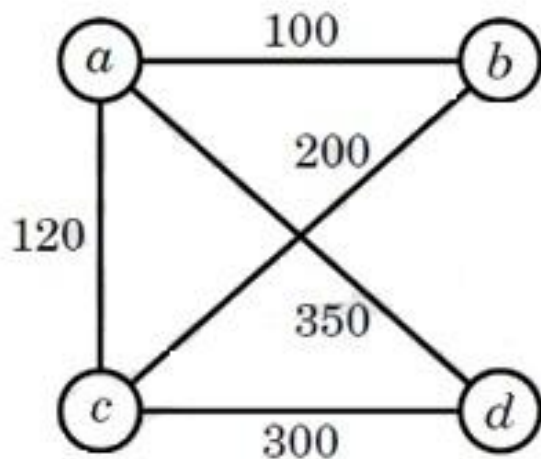
1. Minimalno povezujuće stablo (Primov algoritam)

Težinski grafovi

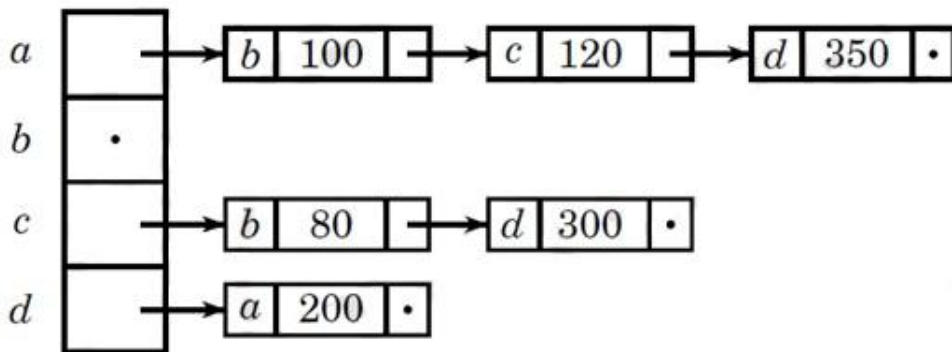
- Za nalaženje najkraćeg puta između dva čvora u povezanom grafu može koristiti BFS algoritam pretrage u širinu, što podrazumeva da su sve grane jednake težine, što u realnim situacijama često nije slučaj
- Zato se koriste **težinski grafovi** (usmereni ili neusmereni) kod kojih svaka grana ima pridruženu težinu, odnosno numeričku vrednost koja može označavati dužinu, vreme, cenu, brzinu, mrežni protok ili neki drugi parametar
- Težinski graf se formalno označava kao $G=(V,E)$ pri čemu svaka grana $e \in E$ ima pridruženu težinu $w(e)$
- Načini predstavljanja težinskih grafova (matrica susedstva i lista susedstva) mogu se lako prilagoditi, u matrici se elementi umesto 0 i 1 zamenjuju konkretnim težinama grana, a u listama susedstva težina se pridružuje svakom povezanom čvoru liste

Težinski grafovi

- Ako neka grana ne postoji u grafu, odgovarajući element matrice susedstva mora biti neka vrednost koja nije moguća za tražene grane, pa se za postojanje nepostojećih grana grafa koristi simbol beskonačnosti (∞ ili oznaka **inf**) ili tačka (•)



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	∞	100	120	350
<i>b</i>	∞	∞	∞	∞
<i>c</i>	∞	80	∞	300
<i>d</i>	200	∞	∞	∞



Minimalno povezujuće stablo

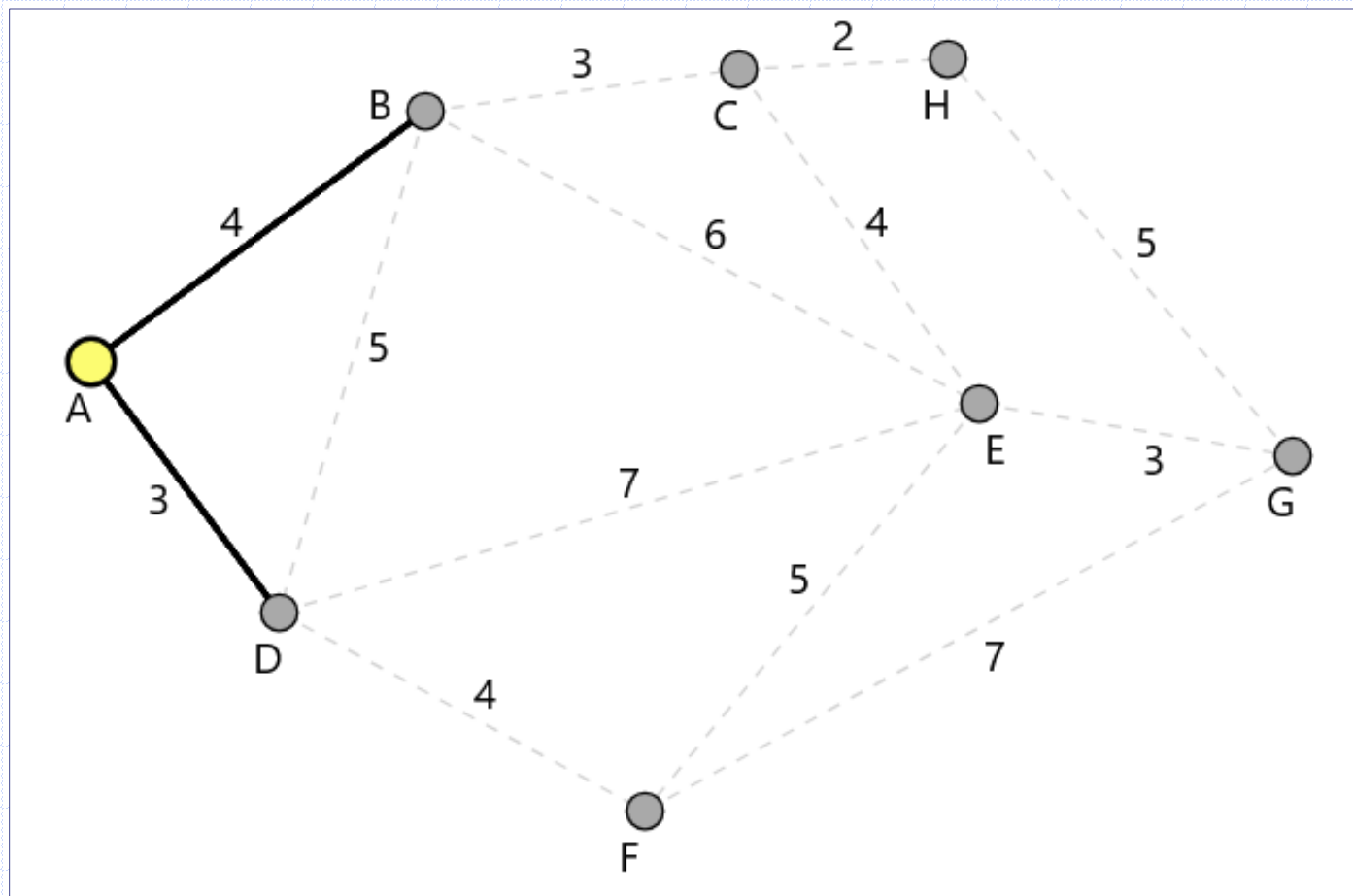
- Težinski grafovi se često koriste za rešavanje problema koji zahtevaju ekonomsko ili optimalno povezivanje svih čvorova (npr. ako kompanija želi da poveže više svojih predstavništva, cilj je pronaći skup veza koji omogućava da sva predstavništva budu povezana, ali uz minimalnu ukupnu cenu)
- Takav podgraf naziva se minimalno povezujuće stablo **MST** (Minimum Spanning Tree) ili „minimalno razapinjujuće stablo“
- Određivanje minimalnog povezujućeg stabla obično se vrši primenom tzv. „gramzivih algoritama“ (**Greedy Algorithms**) a najpoznatiji algoritmi ove klase su **Kruskalov** i **Primov algoritam**
- **Primov algoritam** je prvi objavio češki matematičar Vojteh Jarnik 1930, ponovo ga je objavio Robert C. Prim 1957, a koristio ga je i Edsger V. Dijkstra 1959, pa se ovaj algoritam ponekad naziva i „Jarnikov algoritam“ ili „Prim-Jarnikov algoritam“

Minimalno povezujuće stablo

- Minimalno povezujuće stablo je podgraf datog težinskog grafa koji:
 - Povezuje sve čvorove grafa
 - Ne sadrži nijedan ciklus
 - Ima najmanju moguću sumu težina svojih grana
- MST je povezan, acikličan i najjeftiniji mogući podgraf
- Za pronalaženje MST stabla koriste se efikasni algoritmi koji se oslanjaju na svojstvo podele grafa, tj. ako se skup čvorova grafa podeli na dva dela, svako minimalno stablo mora sadržati granu najmanje težine koja povezuje te delove. Ovo svojstvo je osnova za algoritme kao što su Kruskalov i Primov, koji omogućavaju formiranje minimalnog stabla težinskog grafa
- **Primov algoritam:** U svakom koraku održava se skup čvorova V_k i minimalno povezujuće stablo tog skupa, skup se dalje proširuje najkraćom granom koja povezuje neki čvor grafa koji pripada skupu V_k i neki čvor grafa koji ne pripada skupu V_k . Razlika u odnosu na Dijkstra algoritam je što se ne razmatra rastojanje neoznačenih čvorova od početnog čvora v , već rastojanje od tekućeg skupa V_k . Prilikom dodavanja nekog čvora w skupu V_k , ažuriraju se samo rastojanja čvorova susednih čvoru w (rastojanja ostalih čvorova od skupa V_k se ne menjaju)

Primov algoritam

Primer 1: Primov algoritam počinje razvojem minimalnog povezujućeg stablea (MST) iz nasumičnog čvora, u ovom primeru je čvor **A** izabran kao početni čvor

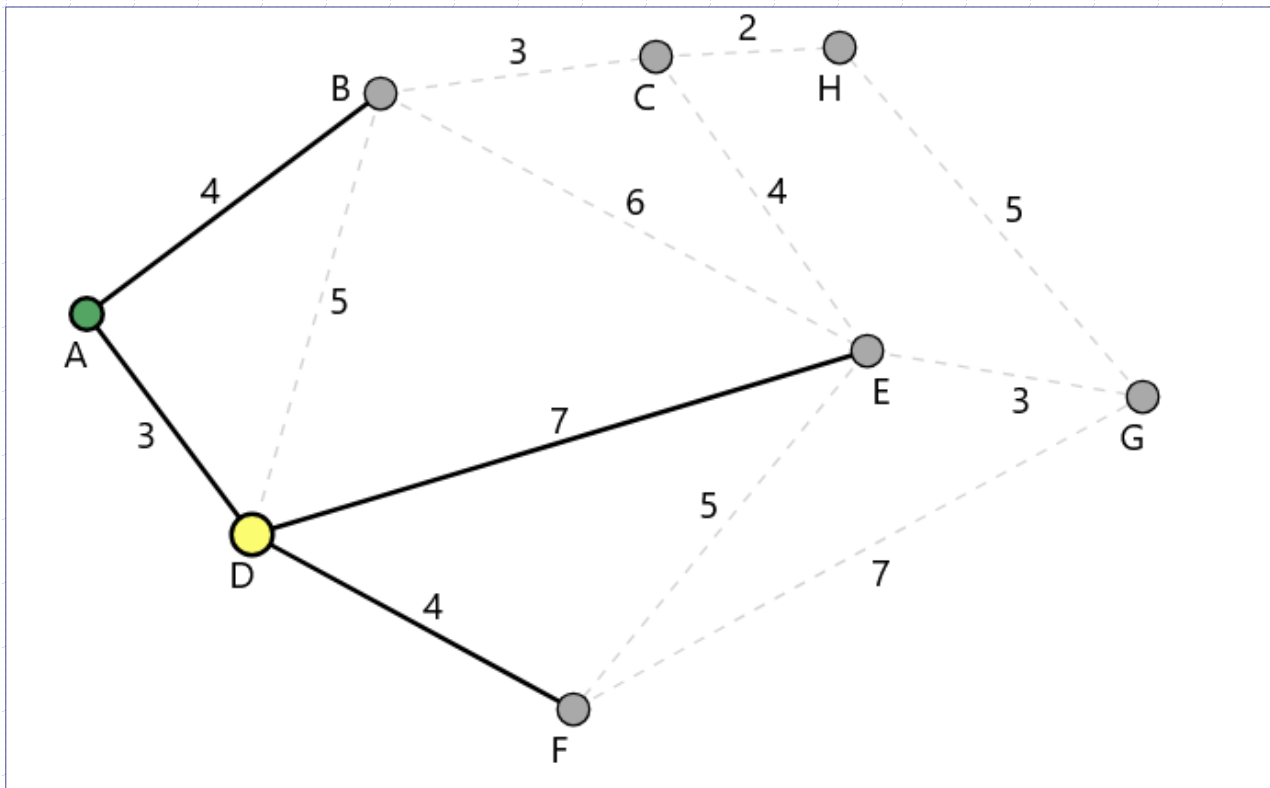


Primov algoritam

Primer 1: Od čvora **A**, MST se širi duž grane sa najmanjom težinom. Čvorovi **A** i **D** sada su u grupi čvorova koji pripadaju MST stablu.

Niz **parent** je ključan za širenje Primovog algoritma u MST stablu. Dalje će niz **parent** biti sledeći:

```
parents = [-1, 0, -1, 0, 3, 3, -1, -1]  
#vertices [ A, B, C, D, E, F, G, H]
```



Primov algoritam

Primer 1:

Niz `parent` je ključan za širenje Primovog algoritma u MST stablu. Dalje će niz `parent` biti sledeći:

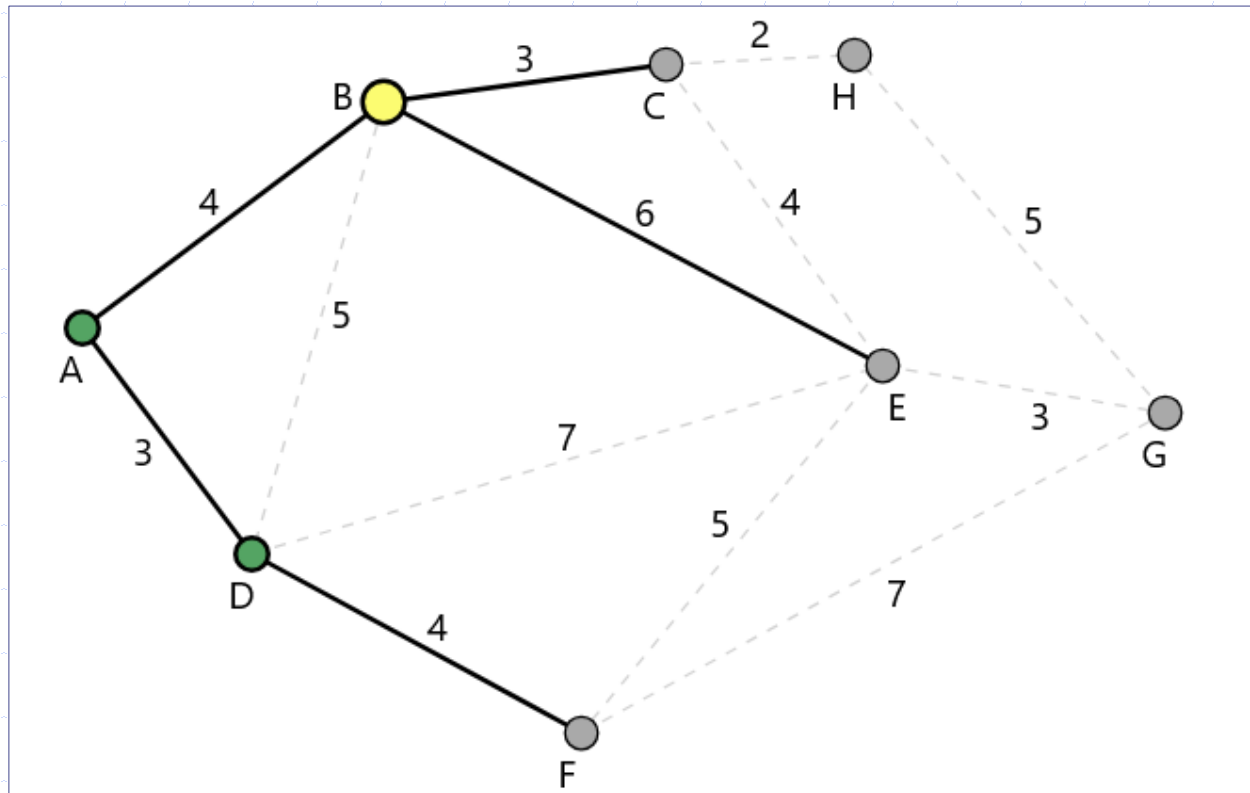
```
parents = [-1, 0, -1, 0, 3, 3, -1, -1]
#vertices [ A,  B,  C,  D,  E,  F,  G,  H]
```

- Čvor **A** je početni čvor, nema roditeljski čvor, u nizu `parents` imaće vrednost -1
- Roditelj čvora **D** je **A**, pa će u nizu čvor **D** imati vrednost 0 (čvor **A** je na indeksu 0)
- Roditelj čvora **B** je takođe **A**, **D** je roditelj za čvorove **E** i **F**
- Niz `parents` se koristi da se održi struktura MST stabla (čvor može imati samo jednog roditelja)
- Da bi se izbegli ciklusi i da bi se pratilo koji su čvorovi trenutno u MST stablu, koristi se niz `in_mst` koji u ovoj fazi ima sledeću strukturu (čvorovi **A** i **D** su **TRUE**)

```
in_mst = [ true, false, false, true, false, false, false, false]
#vertices [  A,    B,    C,    D,    E,    F,    G,    H]
```

Primov algoritam

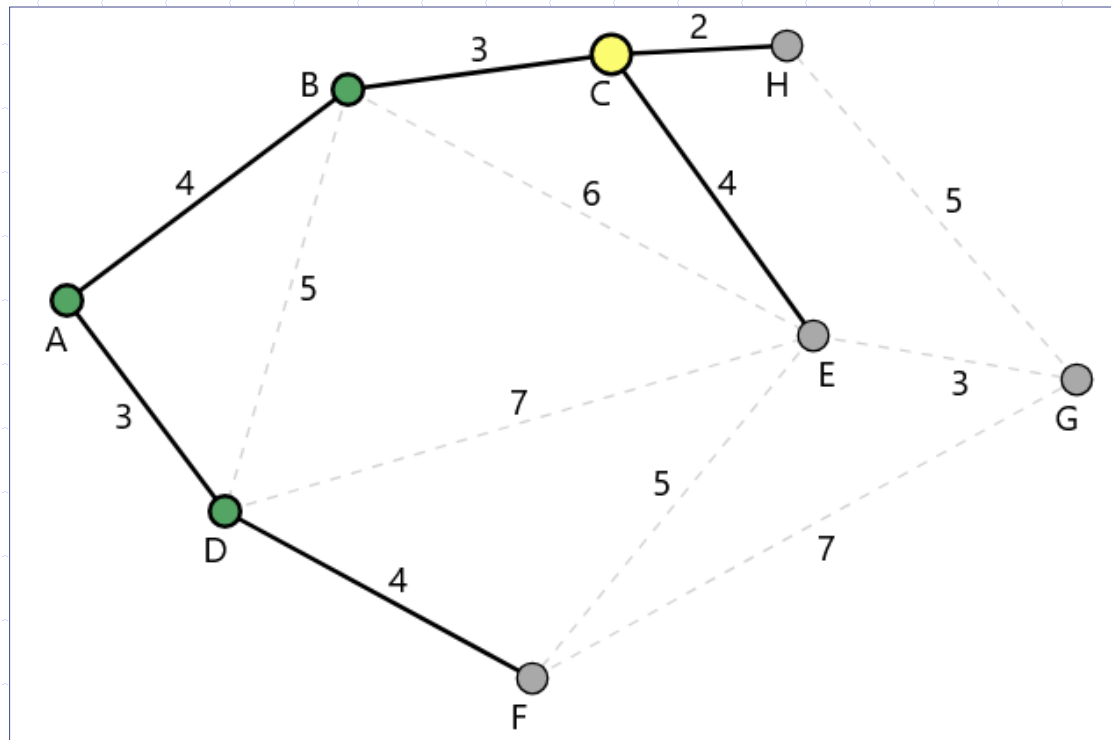
Primer 1: Sledeći korak algoritma je izbor sledećeg čvora kao dela MST stabla, pa se bira čvor najbliži trenutnim MST čvorovima **A** i **D**. Pošto i **A-B** i **D-F** imaju iste težine grana **4**, **B** ili **F** mogu biti izabrani kao sledeći MST čvor, i bira se da to bude čvor **B**. MST grana ka **E** (koja je ranije dolazila iz čvora **D**), sada dolazi iz čvora **B**, jer grana **B-E** ima manju težinu **6** od grane **D-E** sa težinom **7**. Čvor **E** može imati samo jednog roditelja u MST stablu (i u nizu **parents**), tako da **B-E** i **D-E** ne mogu oba biti MST grane ka **E**, pa je to grana **B-E**



Primov algoritam

Primer 1: Sledeći čvor u MST stablu će biti čvor **C**, jer je grana **B-C** sa težinom **3**, grana najmanje težine od svih trenutnih MST čvorova.

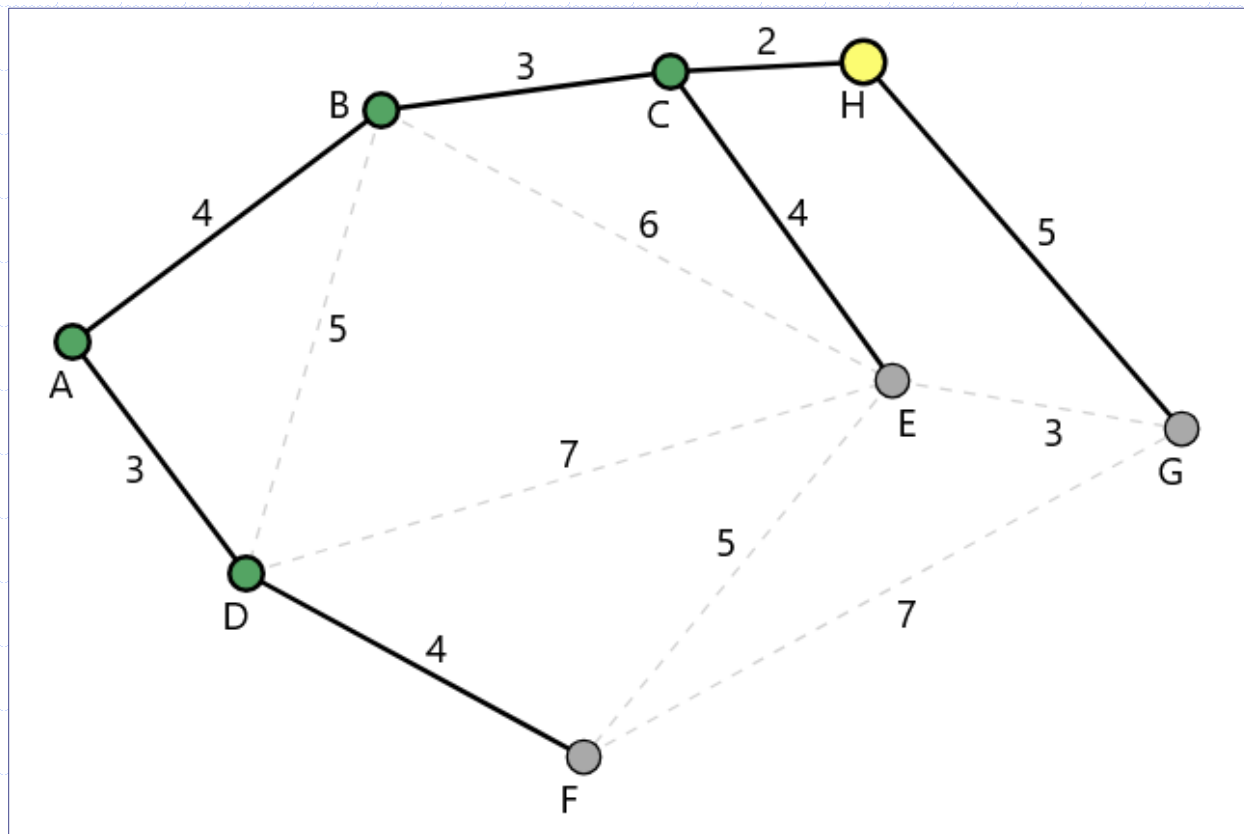
Nakon uključenje čvora **C** u MST stablo, vrši se provera težina grana iz **C** da bi se utvrdilo da li postoje grane manje težine od **C** do čvorova van MST stabla. Grana **C-E** ima manju težinu (4) od prethodne **B-E** MST grane (**6**), a grana **C-H** se uključuje u MST stablo sa težinom grane **2**



Primov algoritam

Primer 1: Čvor **H** je sledeći koji se uključuje u MST stablo, jer ima najmanju težinu grane **6**, a čvor **H** postaje roditelj čvora **G** u nizu **parents**

Sledeći čvor koji treba uključiti u MST stablo je **E** ili **F** jer oba imaju težinu grana **4**, bira se da to bude čvor **E**

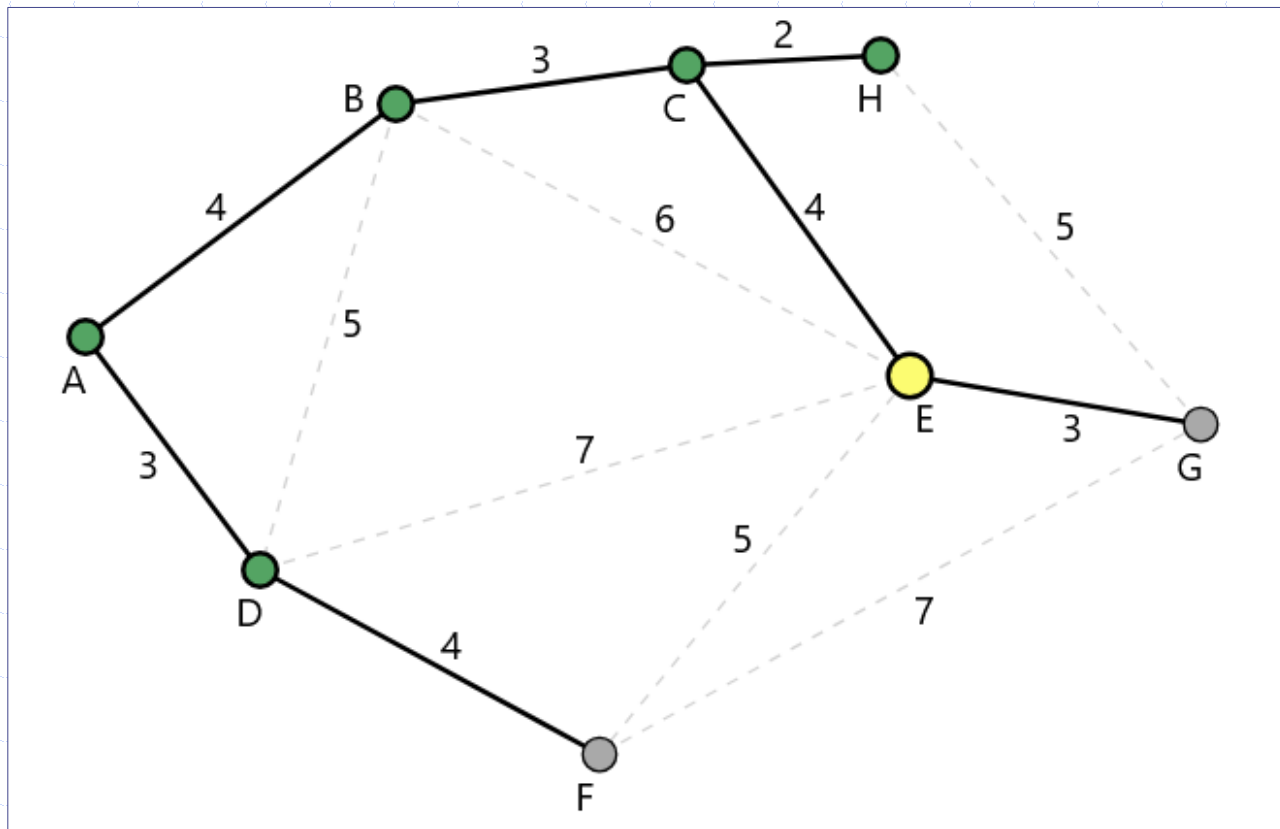


Primov algoritam

Primer 1: Čvor **E** je sledeći koji se uključuje u MST stablo.

Poslednja dva čvora koja se dodaju u MST stablo su čvorovi **F** i **G**.

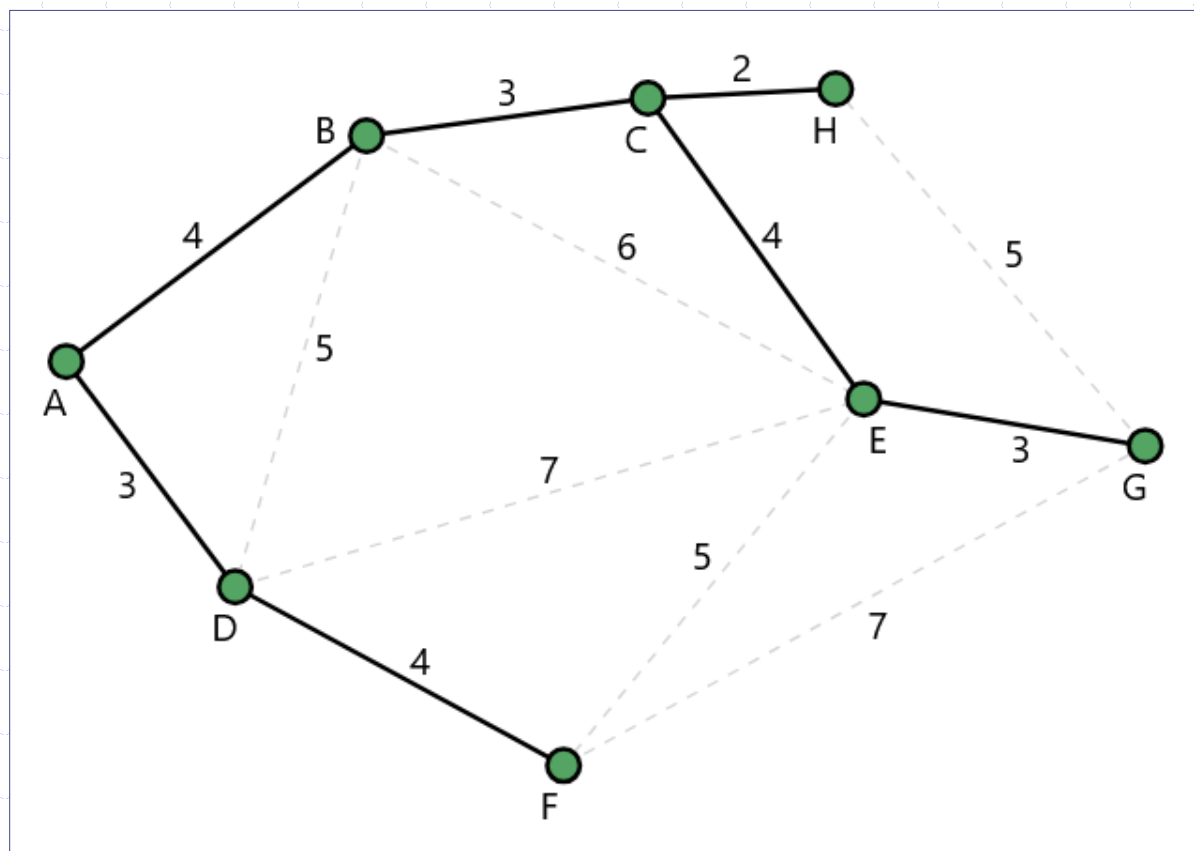
D-F je MST grana ka čvoru **F**, a **E-G** je grana ka čvoru **G**, jer su ove grane sa najmanjom težinom iz trenutnog MST čvora **E**



Primov algoritam

Primer 1:

MST stablo koje je dobijeno upotrebom Primovog algoritma, kako krajnji rezultat prikazuje kolekciju grana u datom grafu, koje povezuju sve čvorove, sa minimalnim zbirom težina grana



Primov algoritam – programski kôd

```
1 class Graph:
2     def __init__(self, size):
3         self.adj_matrix = [[0] * size for _ in range(size)]
4         self.size = size
5         self.vertex_data = [''] * size
6
7     def add_edge(self, u, v, weight):
8         if 0 <= u < self.size and 0 <= v < self.size:
9             self.adj_matrix[u][v] = weight
10            self.adj_matrix[v][u] = weight # Za neusmeren graf
11
12    def add_vertex_data(self, vertex, data):
13        if 0 <= vertex < self.size:
14            self.vertex_data[vertex] = data
15
16    def prims_algorithm(self):
17        in_mst = [False] * self.size
18        key_values = [float('inf')] * self.size
19        parents = [-1] * self.size
20
21        key_values[0] = 0 # Start čvor
22
23        print("Grana \tTežina")
24        for _ in range(self.size):
25            u = min((v for v in range(self.size) if not in_mst[v]), key=lambda v: key_values[v])
26
27            in_mst[u] = True
28
29            if parents[u] != -1: # Preskoči print prvog čvora jer je "no parent"
30                print(f"{self.vertex_data[parents[u]]}-{self.vertex_data[u]} \t{self.adj_matrix[u][parents[u]]}")
31
32            for v in range(self.size):
33                if 0 < self.adj_matrix[u][v] < key_values[v] and not in_mst[v]:
34                    key_values[v] = self.adj_matrix[u][v]
35                    parents[v] = u
```


Primov algoritam – programski kôd

```
36
37 g = Graph(8)
38
39 g.add_vertex_data(0, 'A')
40 g.add_vertex_data(1, 'B')
41 g.add_vertex_data(2, 'C')
42 g.add_vertex_data(3, 'D')
43 g.add_vertex_data(4, 'E')
44 g.add_vertex_data(5, 'F')
45 g.add_vertex_data(6, 'G')
46 g.add_vertex_data(7, 'H')
```

```
47
48 g.add_edge(0, 1, 4) # A - B
49 g.add_edge(0, 3, 3) # A - D
50 g.add_edge(1, 2, 3) # B - C
51 g.add_edge(1, 3, 5) # B - D
52 g.add_edge(1, 4, 6) # B - E
53 g.add_edge(2, 4, 4) # C - E
54 g.add_edge(2, 7, 2) # C - H
55 g.add_edge(3, 4, 7) # D - E
56 g.add_edge(3, 5, 4) # D - F
57 g.add_edge(4, 5, 5) # E - F
58 g.add_edge(4, 6, 3) # E - G
59 g.add_edge(5, 6, 7) # F - G
60 g.add_edge(6, 7, 5) # G - H
```

```
61
62 print("Primov MST Algoritam:")
63 g.prims_algorithm()
```

Primov MST Algoritam:

Grana Težina

A-D 3

A-B 4

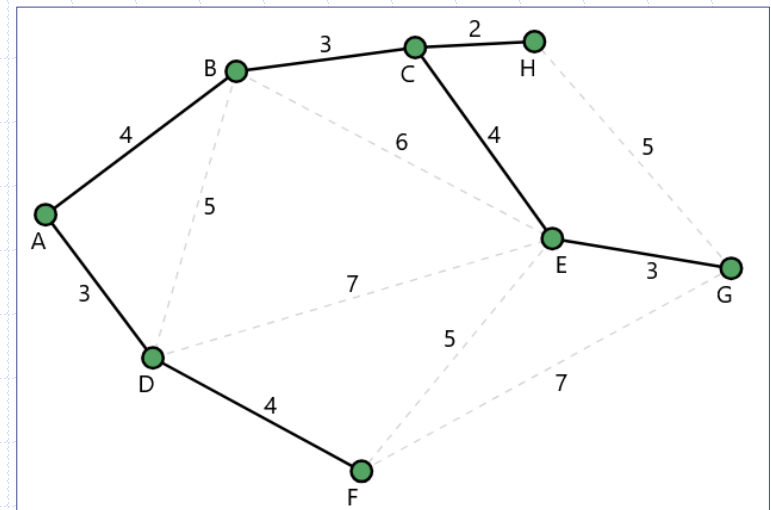
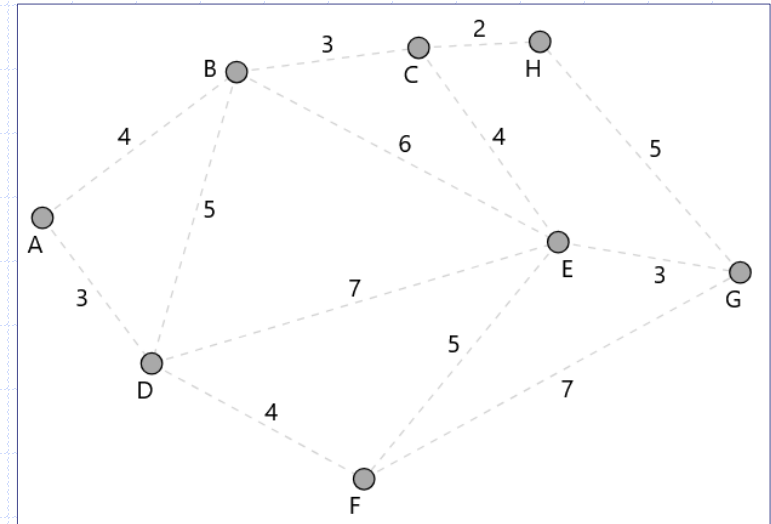
B-C 3

C-H 2

C-E 4

E-G 3

D-F 4

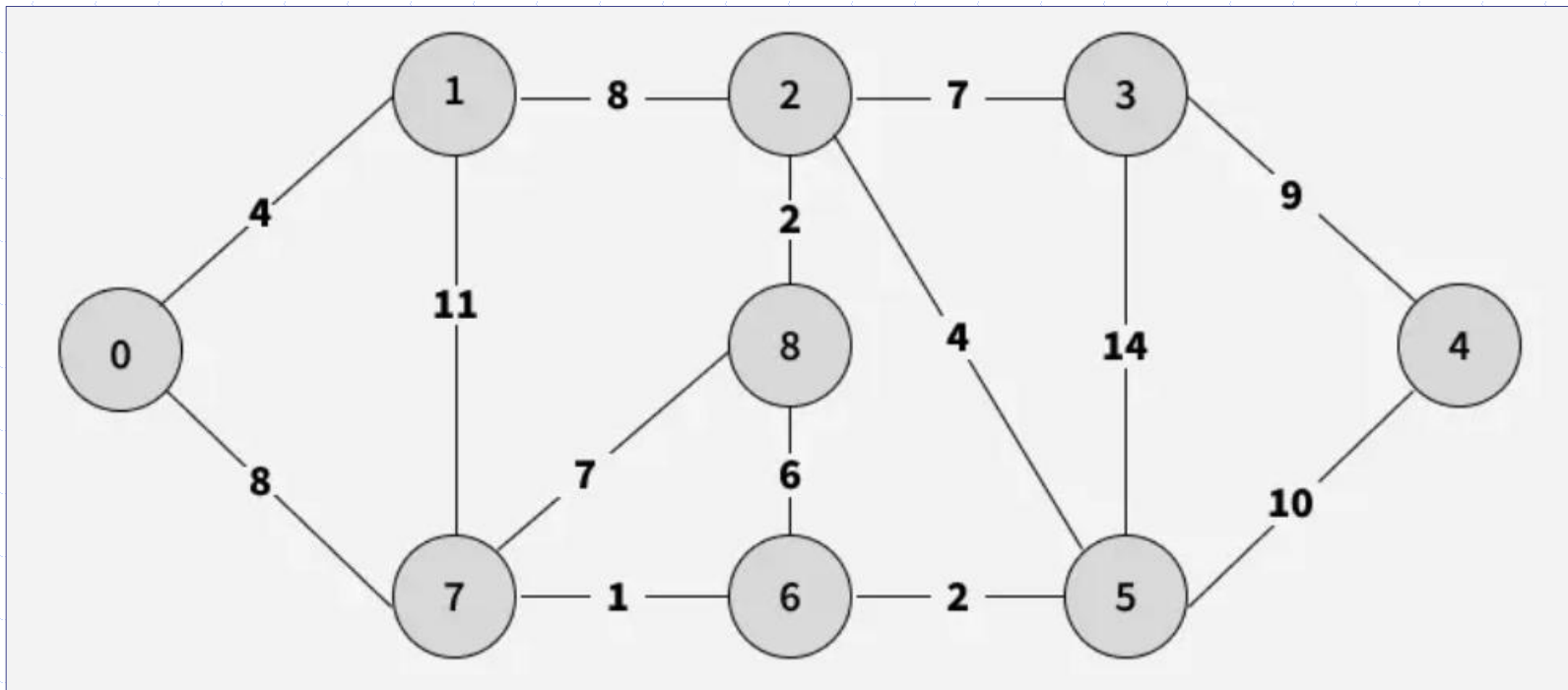


Primov algoritam – programski kôd

- Linija 3-5: Na početku je matrica susednosti prazna, u grafu nema grana, čvorovi nisu obeleženi
- Linija 7-10: Funkcija `add_edge` služi za dodavanje grane, sa vrednošću težine grane za neusmeren grafu
- Linija 12-14: Funkcija `add_vertex_data` dodaje imena čvorovima, npr. 'A' ili 'B'
- Nakon kreiranja elemenata grafa, vrši se implementacija Primovog algoritma preko funkcije `prims_algorithm` unutar klase `Graph`
- Linija 17: Niz `in_mst` sadrži status čvorova koji su u MST stablu (na početku, nijedan čvor nije deo MST stabla)
- Linija 18: Niz `key_values` sadrži trenutno najkraće rastojanje od MST čvorova do svakog čvora van MST stabla
- Linija 19: MST grane se čuvaju u nizu `parents`. Svaka MST grana se čuva čuvanjem roditeljskog indeksa za svaki čvor
- Linija 21: Prvi čvor (čvor A sa indeksom 0) je postavljen kao početni čvor. Promena indeksa na 4 će pokrenuti Primov algoritam od čvora E
- Linija 25: Indeks se pronalazi za čvor sa najmanjom vrednošću koji još nije deo MST
- Linija 32-35: Nakon što se novi čvor doda u MST (linija 27), proverava se da li sada postoje grane iz novododatog MST čvora manje težine, kako bi se prešlo na druge čvorove van MST. Ako je to slučaj, nizovi `key_values` i `parents` se ažuriraju

Primov algoritam

Primer 2: Algoritam počinje od jednog čvora i kreće se kroz nekoliko susednih čvorova, kako bi istražile sve povezane grane. Algoritam počinje sa praznim povezujućim stablom. Osnovna ideja je da se održavaju dva skupa čvorova. Prvi skup sadrži čvorove koji su već uključeni u MST, a drugi skup čvorove koji još nisu u MST stablu. U svakom koraku razmatraju se sve grane koje povezuju dva skupa i bira se grana minimalne težine. Nakon izbora grane, pomera se krajnja tačka grane tako da ona postaje deo skupa MST čvorova.

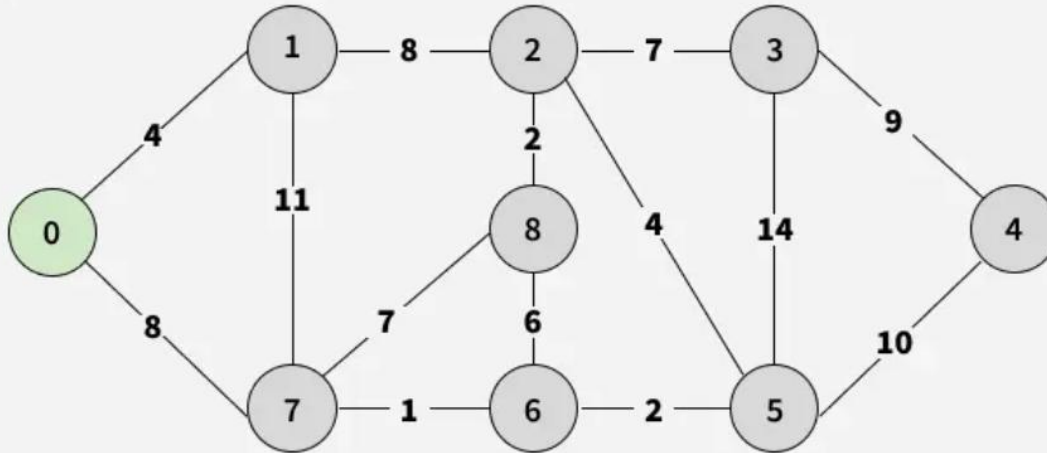


Primov algoritam

Primer 2:

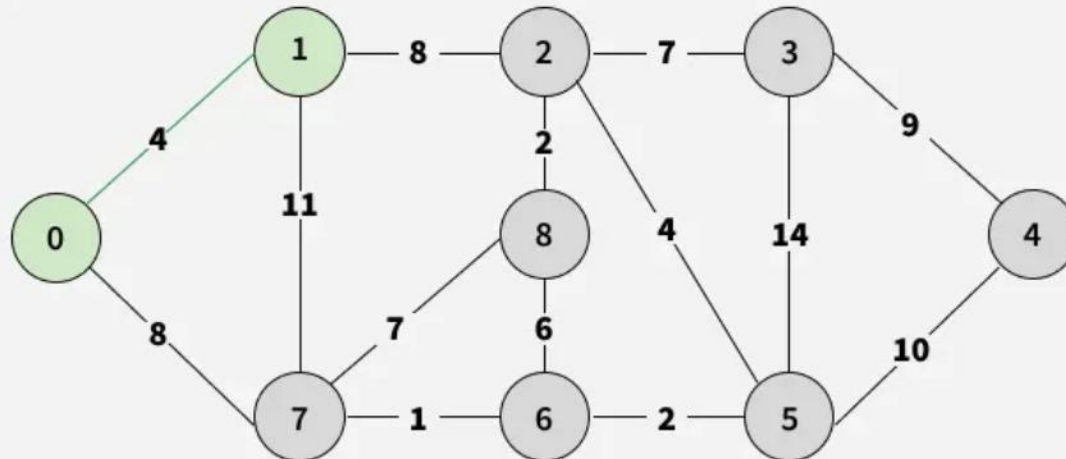
01
Step

Start with edges $\{0,1\}$ and $\{0,7\}$; pick the minimum $\{0,1\}$ and add vertex 1.



02
Step

Choose between $\{0,7\}$ and $\{1,2\}$; pick $\{0,7\}$ (or $\{1,2\}$), adding vertex 7.

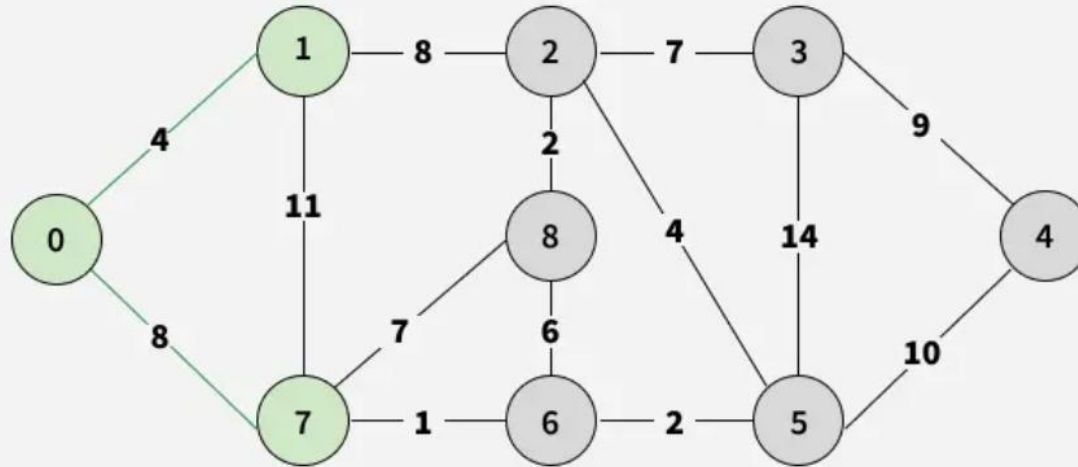


Primov algoritam

Primer 2:

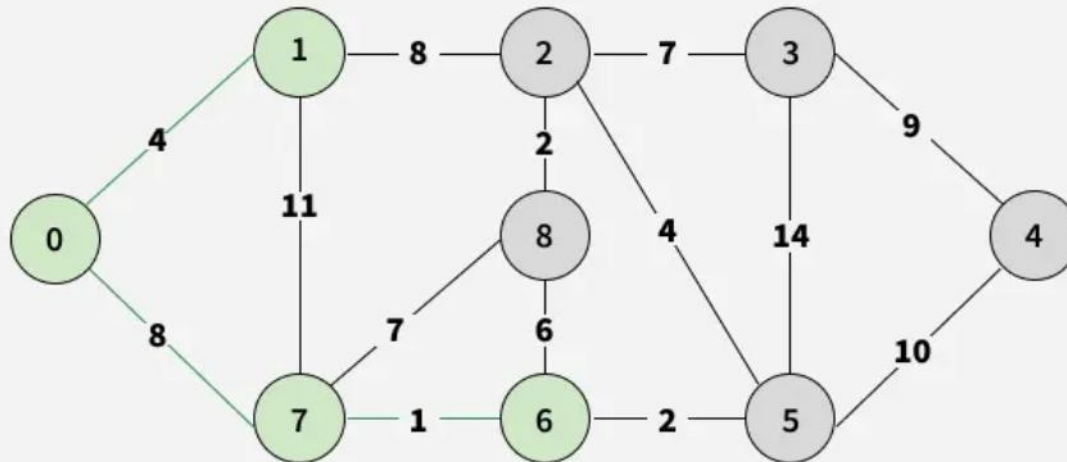
03
Step

Select {7,6} as it has the least weight (1), adding vertex 6.



04
Step

Among {7,8}, {6,8}, and {6,5}, pick {6,5} (weight 2) and add vertex 5.

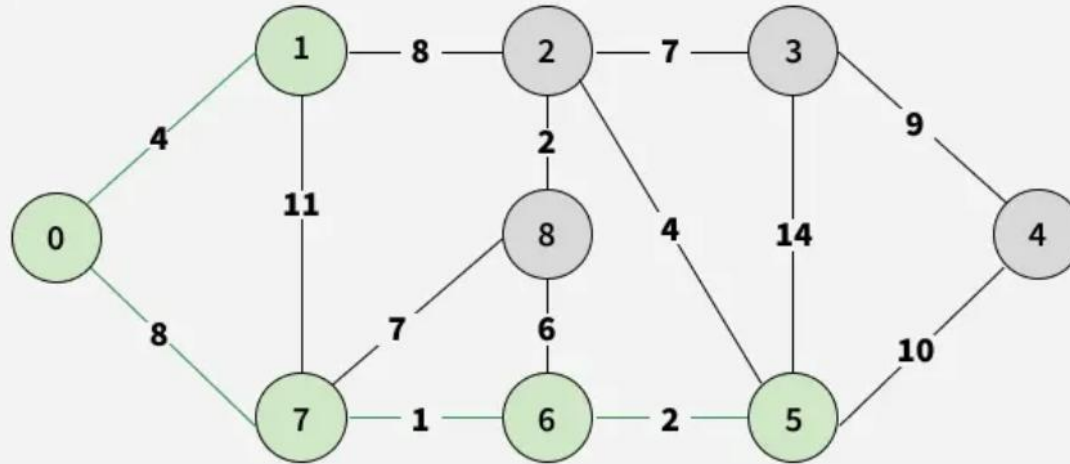


Primov algoritam

Primer 2:

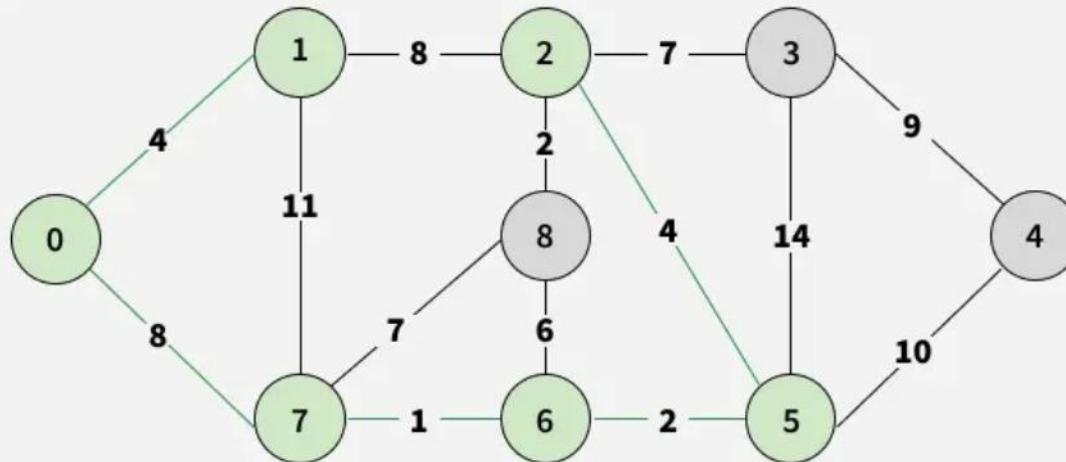
05
Step

Select {5,2} (weight 4), adding vertex 2.



06
Step

Choose {2,8} (weight 2), adding vertex 8.

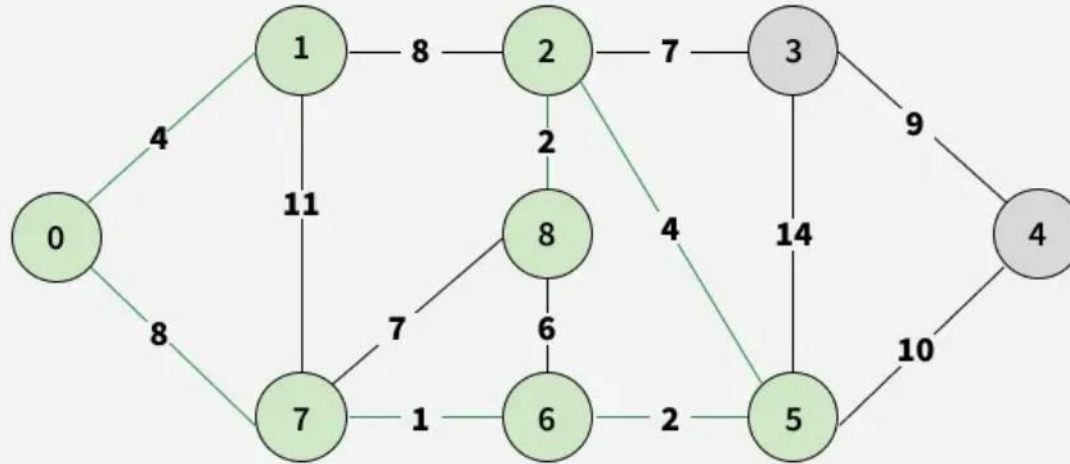


Primov algoritam

Primer 2:

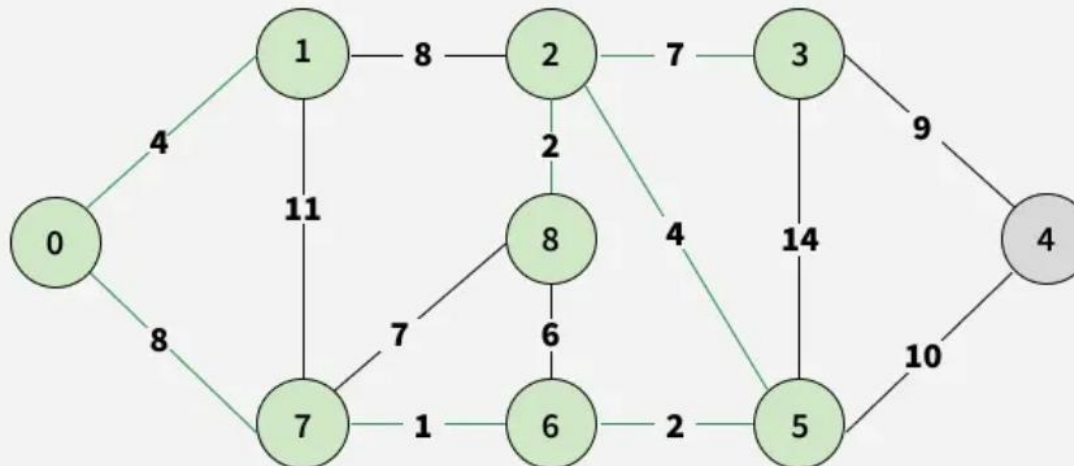
07
Step

Pick {2,3} as the smallest edge, adding vertex 3.



08
Step

Finally, select {3,4}, adding vertex 4.

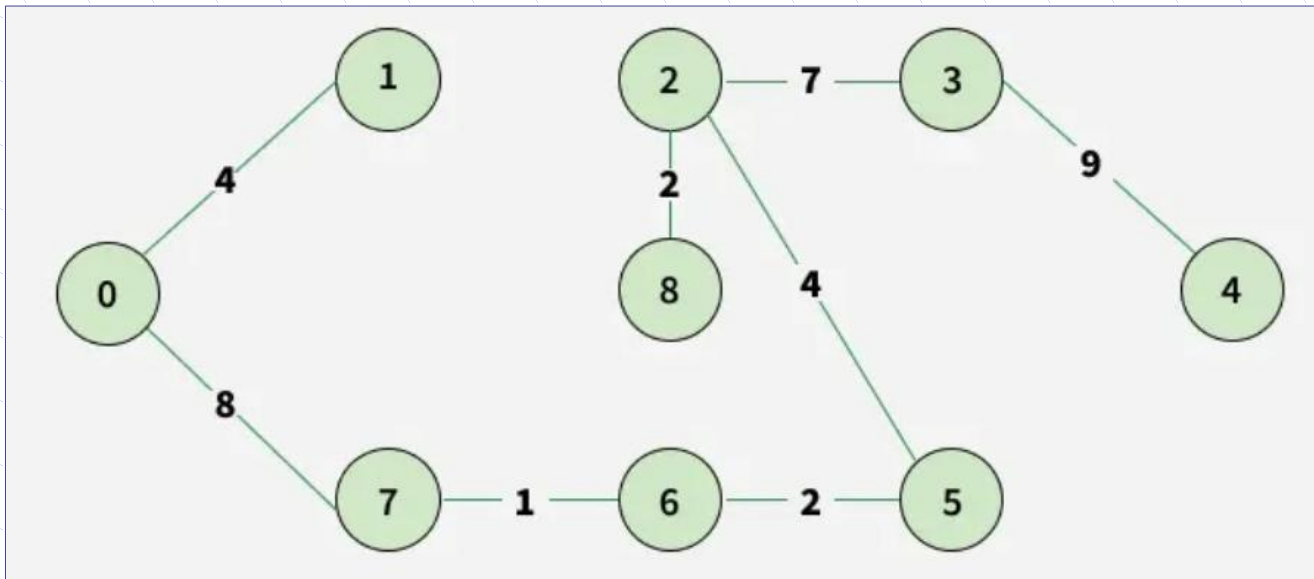
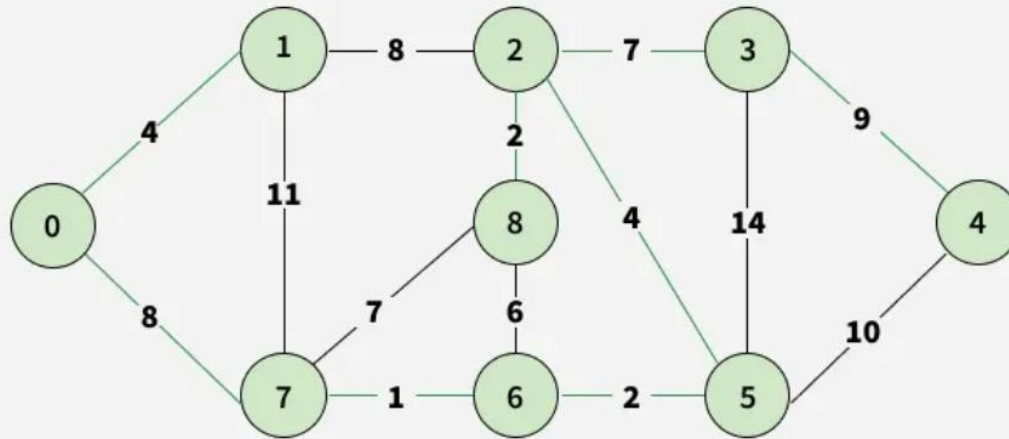


Primov algoritam

Primer 2:

09
Step

All vertices are included, so the MST is complete.



Složenost Primovog algoritma

- Kao rezultat Primovog algoritma dobija se minimalno povezujuće stablo grafa, tj. struktura koja povezuje sve čvorove uz najmanju ukupnu sumu težine grana, bez formiranje ciklusa
- Vremenska složenost Primovog algoritma je $O(V^2)$, gde V označava ukupan broj čvorova grafa
- Ova vremenska složenost je zbog ugnežđenih petlji unutar Primovog algoritma (jedna **for** petlja sa dve druge **for** petlje unutar nje). Prva **for** petlja prolazi kroz sve čvorove grafa i ima složenost $O(V)$. Druga **for** petlja prolazi kroz sve susedne čvorove grafa da bi se našao čvor sa najmanjom vrednošću grane koji je van MST, i ona ima vremensku složenost $O(V)$. Treća **for** petlja proverava sve ostale čvorove kako bi se proverilo da li postoje grane iz novododatog MST čvora koje mogu dovesti do ažuriranja roditeljskih relacija, i ona takođe ima vremensku složenost $O(V)$
- Uzimajući sve tri petlje u obzir, vremenska složenost Primovog algoritma je:

$$O(V) \cdot (O(V) + O(V)) = O(V) \cdot (2 \cdot O(V)) = O(V \cdot 2 \cdot V) = O(2 \cdot V^2) = O(V^2)$$

- Kada se u grafu koriste strukture podataka kao što su Fibonačijev hip, binarni hip ili prioritarni red, tada je vremenska složenost Primovog algoritma $O(E \log V)$ gde V označava ukupan broj čvorova, a E označava ukupan broj grana grafa

2. Najkraći putevi u težinskim grafovima (Dijkstra algoritam)

Najkraći putevi do čvora grafa

- U mnogim primenama nije potrebno povezati sve čvorove, već pronaći najkraći put između dva čvora u težinskom grafu, ili najkraće puteve od jednog čvora do svih ostalih
- Problem određivanja najkraćih puteva uglavnom postoji u navigacionim sistemima, mrežnim protokolima ili planiranju transportnih ruta i za njihovo rešenje koriste se algoritmi za najkraći put, među kojima su najpoznatiji **Dijkstra algoritam**, **Floyd-Varšalov**, **Belman-Fordov algoritam** itd...
- Ako je povezani težinski graf prikazan kao $G=(V,E)$, u vezi sa pronalaženjem najkraćih puteva u grafu G podrazumeva se da numeričke vrednosti pridružene granama grafa predstavljaju njihovu **dužinu**, a ne težinu
- Za svaku granu $e=\{u,v\}$ u grafu G definisana je dužina $p(e) \geq 0$, koja predstavlja meru prelaska u jednom koraku od čvora u do čvora v
- Dužina puta P u grafu predstavlja zbor dužina svih grana koje taj put sadrži

Najkraći putevi do čvora grafa

Dijkstra algoritam:

- Dijkstra algoritam pronalazi najkraće puteve u težinskom grafu bez negativnih dužina grana
- Zasniva se na ideji postepenog širenja skupa rešenih čvorova čiji su najkraći putevi već poznati
- Na početku se formira skup $S=\{s\}$ koji sadrži samo početni čvor. Svim čvorovima se dodeljuje vrednost rastojanja $d(v)$, gde je $d(s)=0$, a za sve ostale čvorove $d(v)=\infty$. U svakom koraku bira se čvor $x \notin S$ koji ima najmanju vrednost $d(x)$. Čvor x se dodaje u skup S , za sve njegove susede v ažurira se rastojanje ako se pronađe kraći put:

$$d(v)=\min(d(v),d(x)+P(x,v))$$

- Ova operacija naziva se popuštanje (relaxing). Postupak se ponavlja dok svi čvorovi ne uđu u skup S
- Dijkstra algoritam (kao i Primov) spada u grupu „pohlepnih algoritama“, ali dok Primov algoritam traži minimalno stablo koje povezuje sve čvorove, Dijkstra traži najkraći put od jednog čvora do svih ostalih u težinskom grafu bez negativnih grana

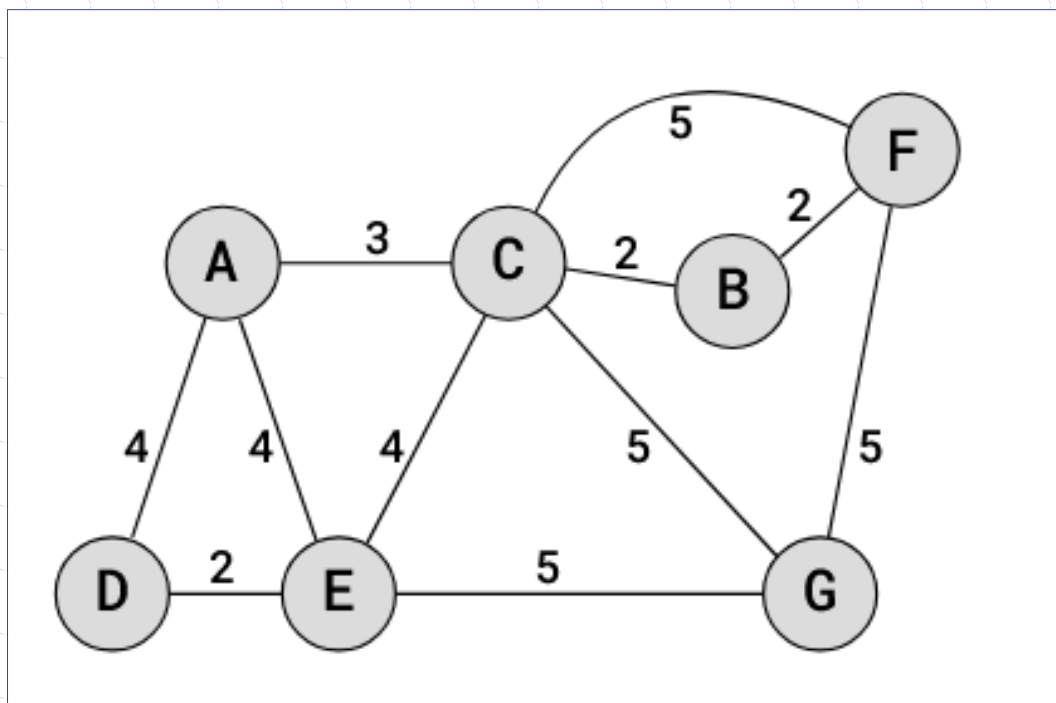
Dijkstra algoritam - pseudokôd

```
function Dijkstra(Graph, source):  
    create distance[] and set all to infinity  
    distance[source] = 0  
    create a priority queue Q  
  
    while Q is not empty:  
        u = vertex in Q with smallest distance  
        remove u from Q  
  
        for each neighbor v of u:  
            alt = distance[u] + weight(u, v)  
            if alt < distance[v]:  
                distance[v] = alt  
                update v in Q with new distance  
  
    return distance[]
```

Algoritam počinje postavljanjem svih rastojanja čvorova na beskonačno, osim za izvorni čvor, koje je nula. Ponavlja se više puta izbor čvora sa najmanjim rastojanjem, a zatim se ažuriraju rastojanja njegovih suseda, ako se pronade kraća putanja. Proces se nastavlja dok se ne posete svi čvorovi, a kao rezultat se dobijaju najkraće putanje od izvora do svih čvorova u grafu.

Dijkstra algoritam

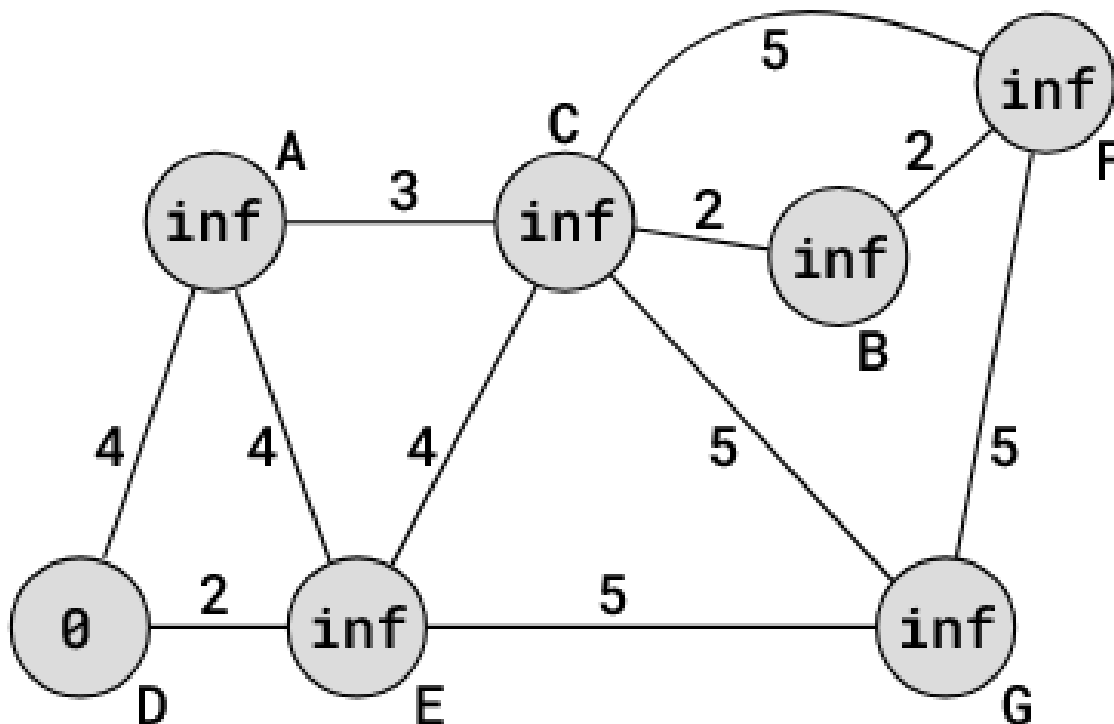
Primer 1: Pronaći najkraći put od izvornog čvora **D** do svih ostalih čvorova, tako da je npr. najkraći put do **C** preko **D->E->C** (težina putanje $2+4=6$). Da bi pronašao najkraći put, Dijkstra algoritam koristi niz sa rastojanjima do svih ostalih čvorova i početno postavlja ta rastojanja na beskonačno (**inf**), dok je rastojanje do početnog čvora **D** postavljeno na vrednost **0**



```
distances = [inf, inf, inf, 0, inf, inf, inf]
#vertices  [ A , B , C , D , E , F , G ]
```

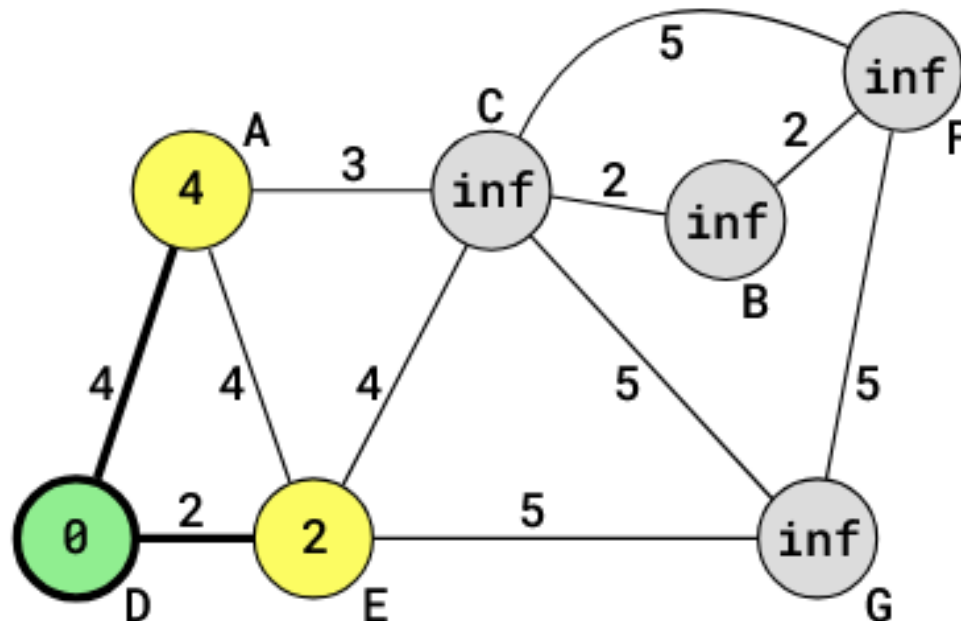
Dijkstra algoritam

Primer 1: Pronaći najkraći put od izvornog čvora **D** do svih ostalih čvorova. Početna rastojanja do svih ostalih čvorova se postavljaju na beskonačno (**inf**), dok je rastojanje do početnog čvora **D** postavljeno na vrednost **0**



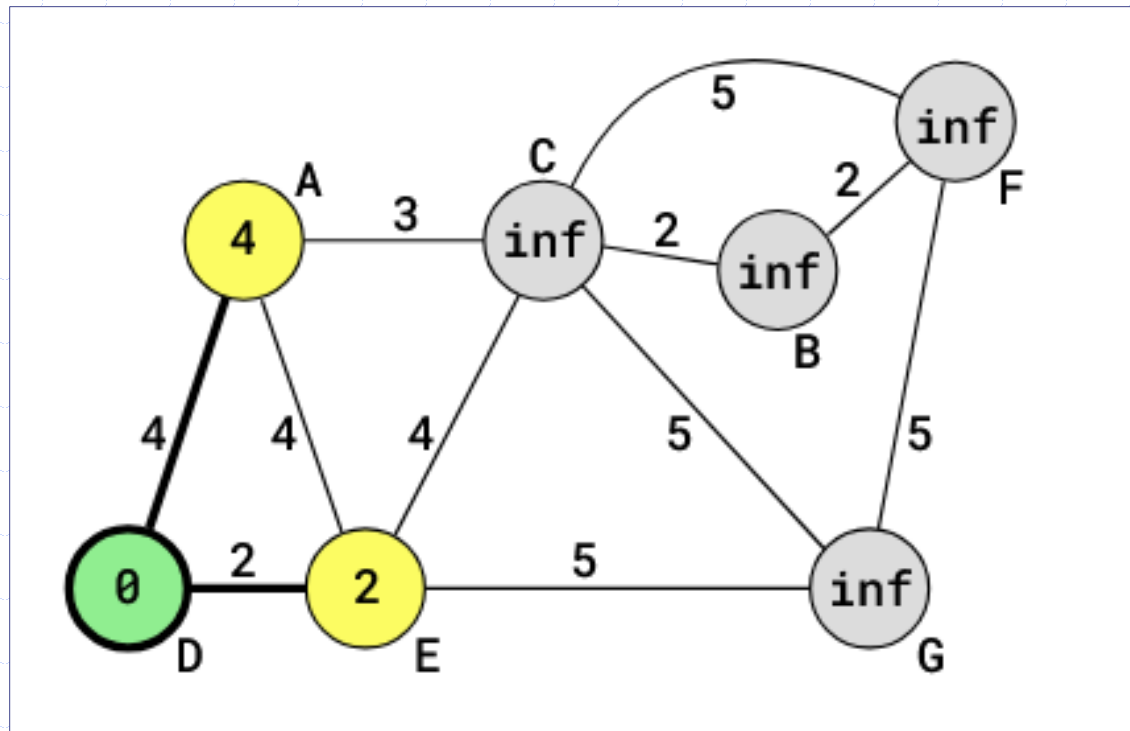
Dijkstra algoritam

Primer 1: Algoritam postavlja čvor **D** kao trenutni čvor i posmatra rastojanje do susednih čvorova. Pošto je početno rastojanje do čvorova **A** i **E** beskonačno, novo rastojanje do njih se ažurira težinama grana pa se rastojanje do čvora **A** menja **sa inf na 4**, a do čvora **E** **sa inf na 2**. Ažuriranje vrednosti rastojanja je faza „relaxing“



Dijkstra algoritam

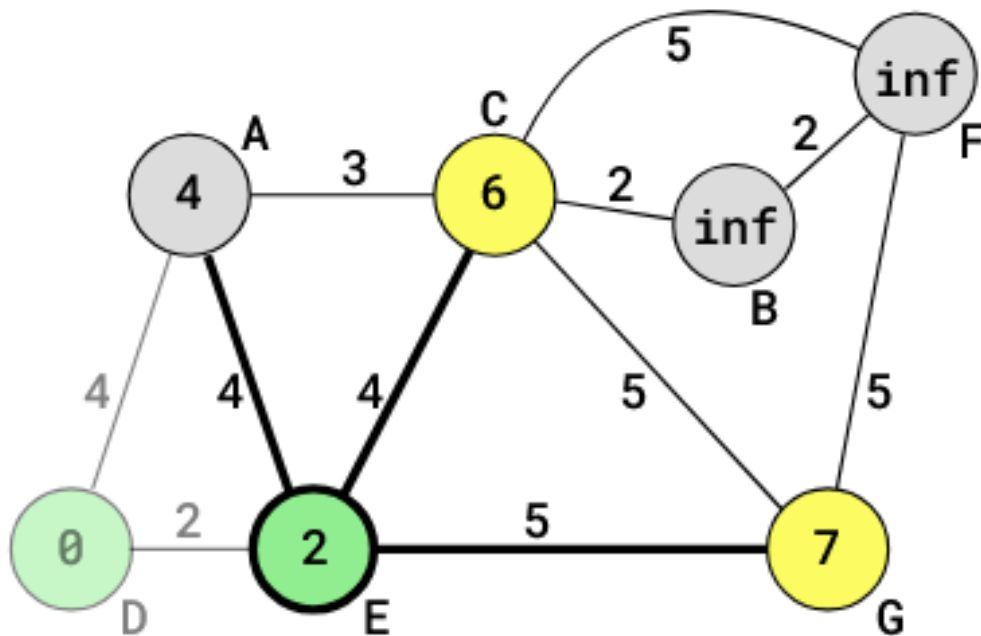
Primer 1: Algoritam postavlja čvor **D** kao trenutni čvor i posmatra rastojanje do susednih čvorova. Pošto je početno rastojanje do čvorova **A** i **E** beskonačno, novo rastojanje do njih se ažurira težinama grana pa se rastojanje do čvora **A** menja **sa inf na 4**, a do čvora **E** **sa inf na 2**, (faza „relaxing“). Nakon relaxing faze za čvorove **A** i **E**, čvor **D** se smatra posećenim i dalje se ne razmatra.



Dijkstra algoritam

Primer 1: Sledeći čvor koji se bira kao trenutni mora biti prethodno neposećeni čvor sa najkraćom udaljenošću od izvornog čvora **D**, pa se čvor **E** bira kao trenutni nakon čvora **D**

Rastojanje od čvora **E** do svih susednih i prethodno neposećenih čvorova se izračunava i ažurira ako je potrebno



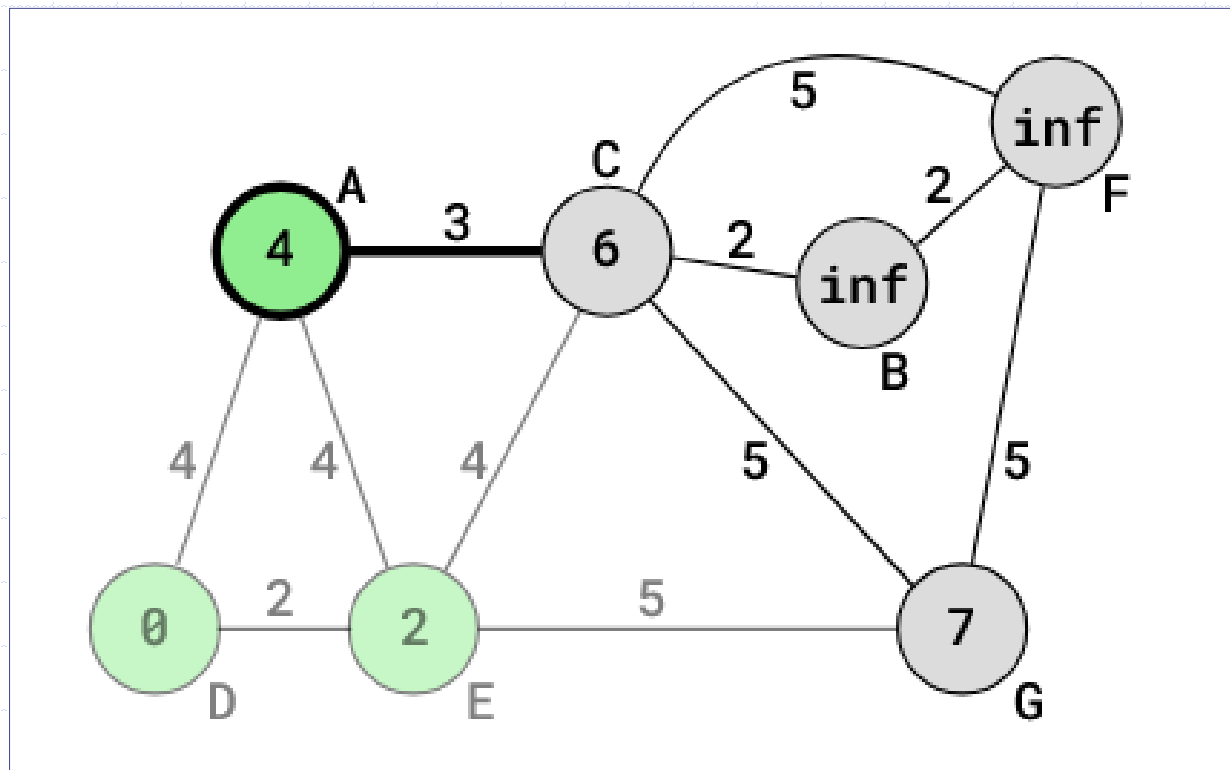
- Izračunato rastojanje od čvora **D** do **A** (preko **E**) je $2+4=6$, ali rastojanje trenutnog čvora **E** do čvora **A** je **4** (što je manje), pa se rastojanje do čvora **A** ne ažurira
- Rastojanje do čvora **C** se izračunava kao $2+4=6$, što je manje od **inf**, pa se rastojanje do čvora **C** ažurira
- Rastojanje do čvora **G** se izračunava i ažurira kao $2+5=7$

Dijkstra algoritam

Primer 1: Sledeći čvor koji treba posetiti je čvor **A** jer ima najkraće rastojanje od **D** od svih neposećenih čvorova

Izračunato rastojanje do čvora **C**, preko **A** je $4+3=7$, što je veće od već podešenog rastojanja **6**, pa se rastojanje do čvora **C** ne ažurira

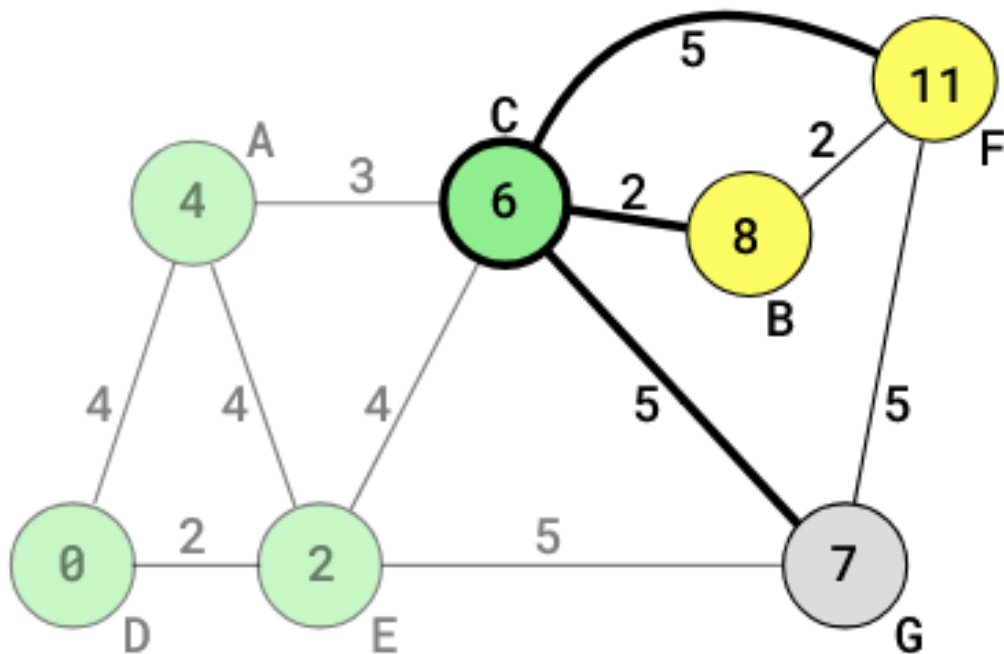
Čvor **A** se zatim označava kao posećen



Dijkstra algoritam

Primer 1: Pošto je čvor **A** je označen kao posećen, sledeći trenutni čvor će biti čvor **C** jer ima najmanje rastojanje od čvora **D** između svih preostalih neposećenih čvorova

Nakon ažuriranja rastojanja do čvorova **F**, **B** i **G**, čvor **C** se označava kao posećen

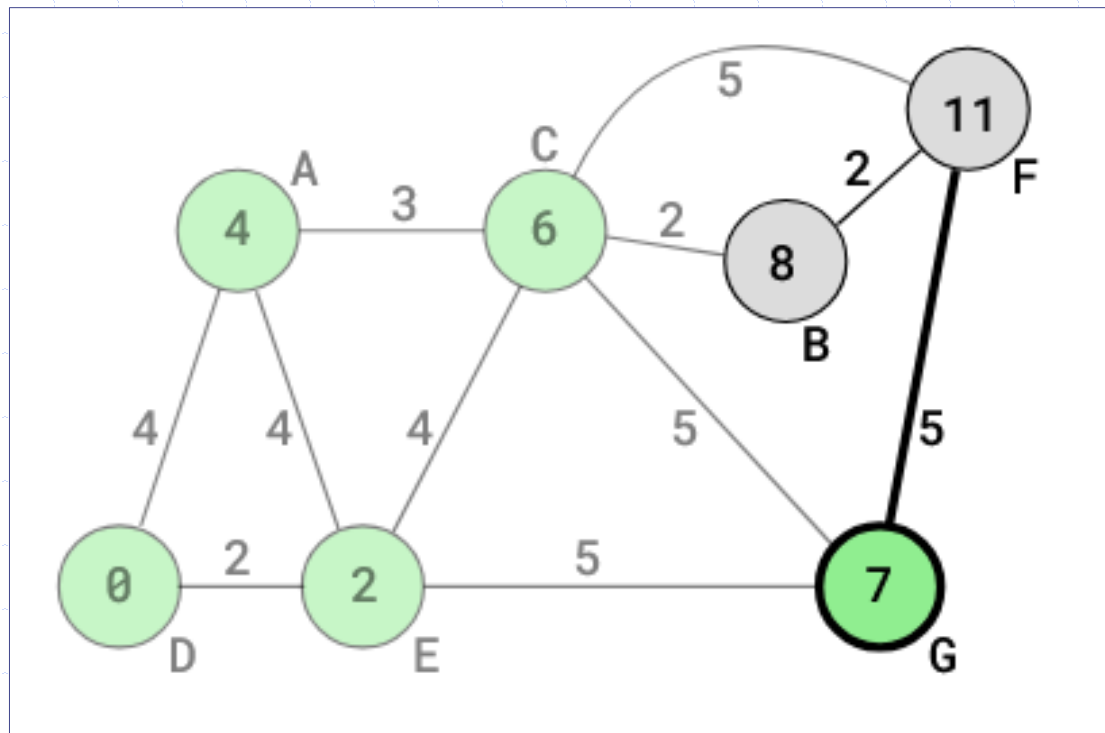


- ▶ Čvor **F** dobija ažurirano rastojanje kao $6+5=11$
- ▶ Čvor **B** dobija ažurirano rastojanje kao $6+2=8$
- ▶ Izračunato rastojanje do čvora **G** (preko čvora **C**) je $6+5=11$, što je veće od već podešenog rastojanja sa vrednošću **7**, tako da se rastojanje do čvora **G** ne ažurira

Dijkstra algoritam

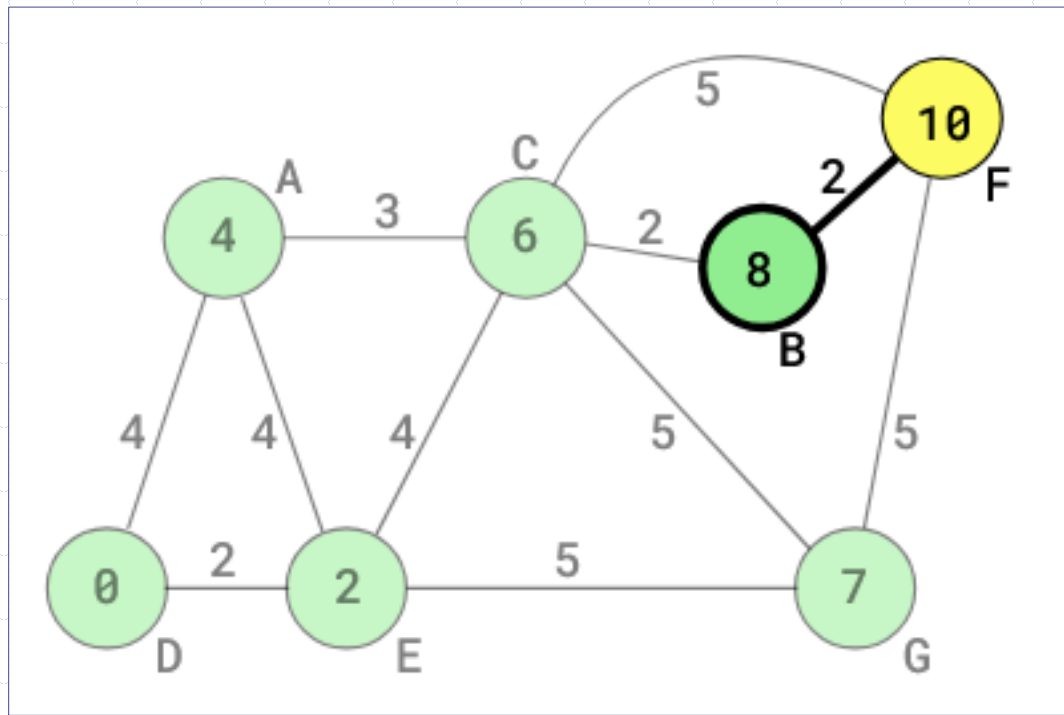
Primer 1: Pošto je čvor **C** već označen kao posećen, sledeći trenutni čvor koji će biti posećen je čvor **G** jer ima najmanje rastojanje od čvora **D** među svim preostalim neposećenim čvorovima

Čvor **F** već ima postavljeno rastojanje na vrednost **11**, što je manje od izračunatog rastojanja od **G**, koje je **7+5=12**, tako da se rastojanje do čvora **F** ne ažurira. Čvor **G** se zatim označava kao posećen



Dijkstra algoritam

Primer 1: Pošto je čvor **G** već označen kao posećen, čvor **B** postaje trenutni čvor jer ima najmanje rastojanje među svim preostalim neposećenim čvorovima. Novo rastojanje do čvora **F** (preko **B**) je $8+2=10$, jer je manje od već postavljenog rastojanja od **11**. Čvor **B** se zatim označava kao posećen, ne proverava se poslednji neposećen čvor **F**, pa je Dijkstra algoritam završen. Svaki čvor je posećen samo jednom, a rezultat je najmanja udaljenost od izvornog čvora **D** do svakog drugog čvora u grafu.



Dijkstra algoritam – programski kôd

```
1 class Graph:
2     def __init__(self, size):
3         self.adj_matrix = [[0] * size for _ in range(size)]
4         self.size = size
5         self.vertex_data = [''] * size
6
7     def add_edge(self, u, v, weight):
8         if 0 <= u < self.size and 0 <= v < self.size:
9             self.adj_matrix[u][v] = weight
10            self.adj_matrix[v][u] = weight # Za neusmeren graf
11
12     def add_vertex_data(self, vertex, data):
13         if 0 <= vertex < self.size:
14             self.vertex_data[vertex] = data
15
16     def dijkstra(self, start_vertex_data):
17         start_vertex = self.vertex_data.index(start_vertex_data)
18         distances = [float('inf')] * self.size
19         distances[start_vertex] = 0
20         visited = [False] * self.size
21
```

generiše se `adj_matrix` da bi se sačuvala sve grane i njihove težine, početne vrednosti su postavljene na 0

`size` predstavlja broj čvorova grafa

`vertex_data` čuva imena svih čvorova

`add_edge` funkcija za povezivanje čvora `u` sa čvorom `v` granom težine `weight`

`add_vertex_data` funkcija za dodavanje čvora grafa, index čvora dat je argumentom `vertex`, dok je `data` ime čvora

`dijkstra` funkcija za pokretanje Dijkstra algoritma

Linije 18 i 19: početno rastojanje za sve čvorove grafa je `inf`, osim za početni čvor gde je 0

Linija 20: svi čvorovi su inicijalno postavljeni na `False`, da bi se u nizu `visited` označili kao neposećeni

Dijkstra algoritam – programski kôd 1

```
22     for _ in range(self.size):
23         min_distance = float('inf')
24         u = None
25         for i in range(self.size):
26             if not visited[i] and distances[i] < min_distance:
27                 min_distance = distances[i]
28                 u = i
29
30         if u is None:
31             break
32
33         visited[u] = True
34
35         for v in range(self.size):
36             if self.adj_matrix[u][v] != 0 and not visited[v]:
37                 alt = distances[u] + self.adj_matrix[u][v]
38                 if alt < distances[v]:
39                     distances[v] = alt
40
41     return distances
```

Linije 23-28: nalaženje neposećenog tekućeg čvora sa najmanjim rastojanjem od početnog čvora

Linije 30-31: Algoritam se prekida ako se ne pronade sledeći susedni čvor

Linija 33: tekući čvor se označava kao posećen pre nego što se rastojanja susednih čvorova ažuriraju

Linije 35-39: rastojanja se izračunavaju samo za neposećene susedne čvorove i ažuriraju se ako su manja od prethodno postavljenih

```
43     g = Graph(7)
44
45     g.add_vertex_data(0, 'A')
46     g.add_vertex_data(1, 'B')
47     g.add_vertex_data(2, 'C')
48     g.add_vertex_data(3, 'D')
49     g.add_vertex_data(4, 'E')
50     g.add_vertex_data(5, 'F')
51     g.add_vertex_data(6, 'G')
52
53     g.add_edge(3, 0, 4) # D - A, weight 5
54     g.add_edge(3, 4, 2) # D - E, weight 2
55     g.add_edge(0, 2, 3) # A - C, weight 3
56     g.add_edge(0, 4, 4) # A - E, weight 4
57     g.add_edge(4, 2, 4) # E - C, weight 4
58     g.add_edge(4, 6, 5) # E - G, weight 5
59     g.add_edge(2, 5, 5) # C - F, weight 5
60     g.add_edge(2, 1, 2) # C - B, weight 2
61     g.add_edge(1, 5, 2) # B - F, weight 2
62     g.add_edge(6, 5, 5) # G - F, weight 5
63
64     # Dijkstra's algorithm from D to all vertices
65     print("\nDijkstra algoritam počinje od čvora D:")
66     distances = g.dijkstra('D')
67     for i, d in enumerate(distances):
68         print(f"Rastojanje od D do {g.vertex_data[i]}: {d}")
```

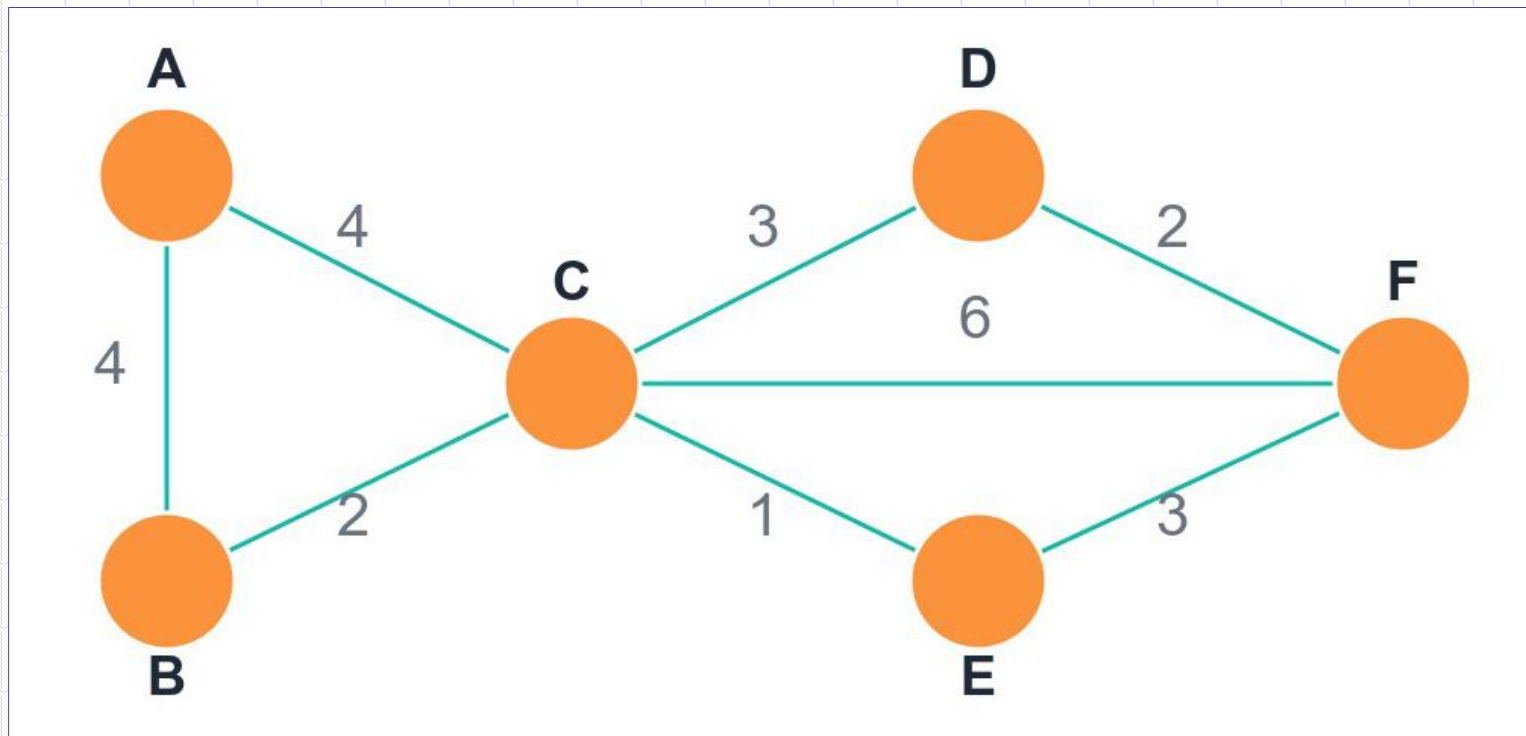


Dijkstra algoritam počinje od čvora D:

Rastojanje od D do A: 4
Rastojanje od D do B: 8
Rastojanje od D do C: 6
Rastojanje od D do D: 0
Rastojanje od D do E: 2
Rastojanje od D do F: 10
Rastojanje od D do G: 7

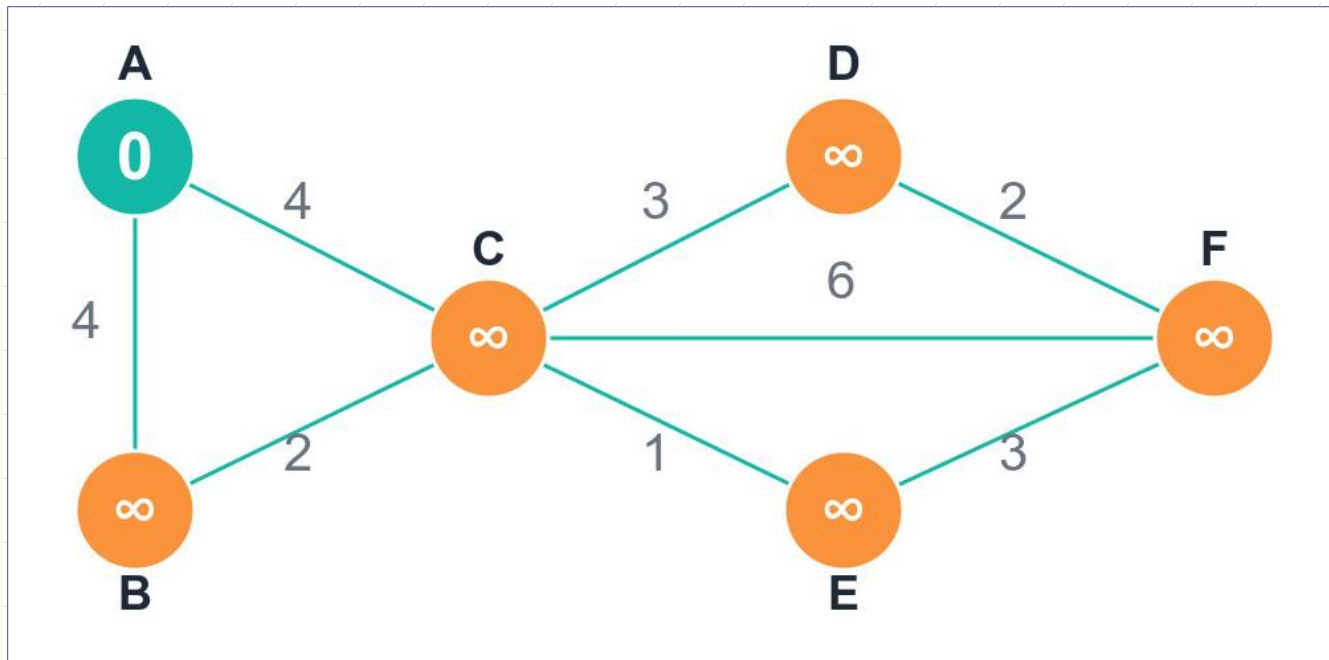
Dijkstra algoritam

Primer 2: Za težinski (ponderisani) graf sa čvorovima A, B, C, D, E i F, i granama koje imaju pozitivne vrednosti težine potrebno je koristeći Dijkstra algoritam pronaći najkraći put od čvora A (izvorni čvor) do svakog drugog čvora u grafu



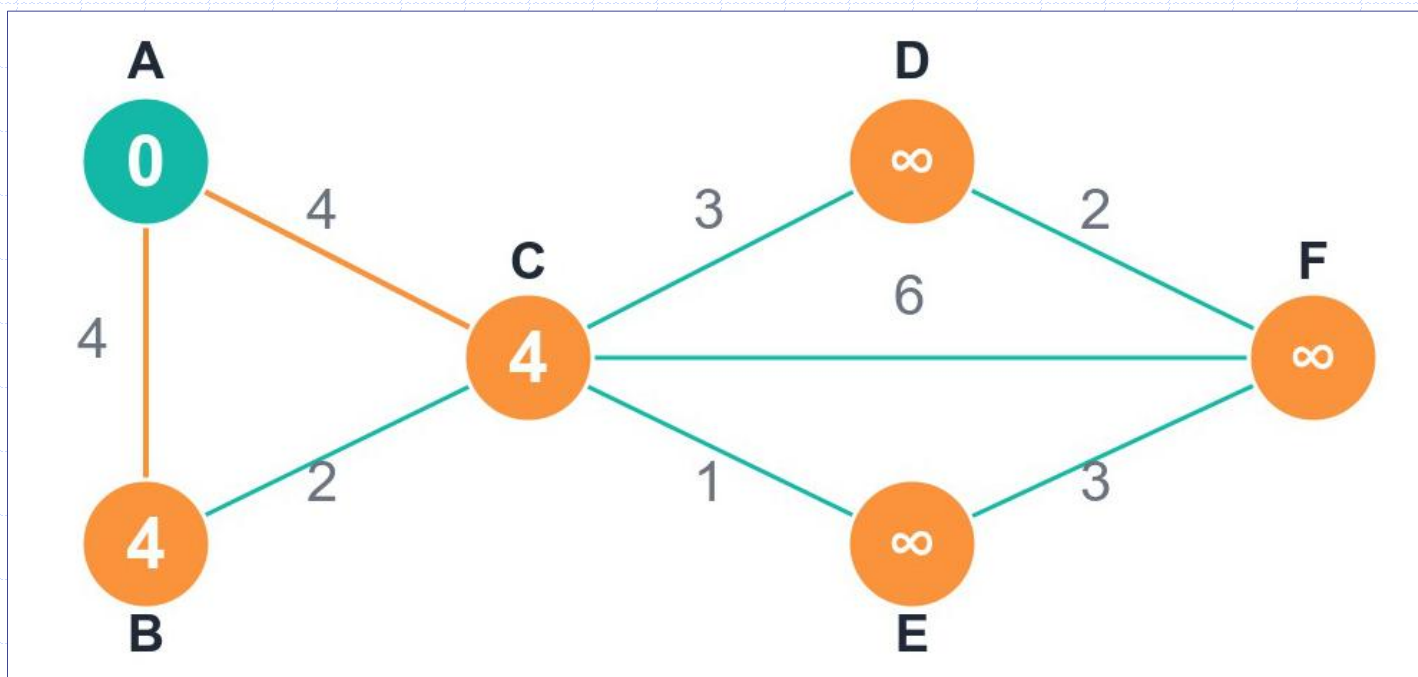
Dijkstra algoritam

Korak 1 - inicijalizacija: Prvo se određuju početna rastojanja od izvornog čvora (A) do svih ostalih čvorova u grafu. Rastojanje do izvornog čvora se postavlja na 0, dok su rastojanja do svih preostalih čvorova (B, C, D, E, F) inicijalizovana na beskonačno (∞), što ukazuje da su ti čvorovi nedostupni. Algoritam počinje procesiranje od izvornog čvora A.



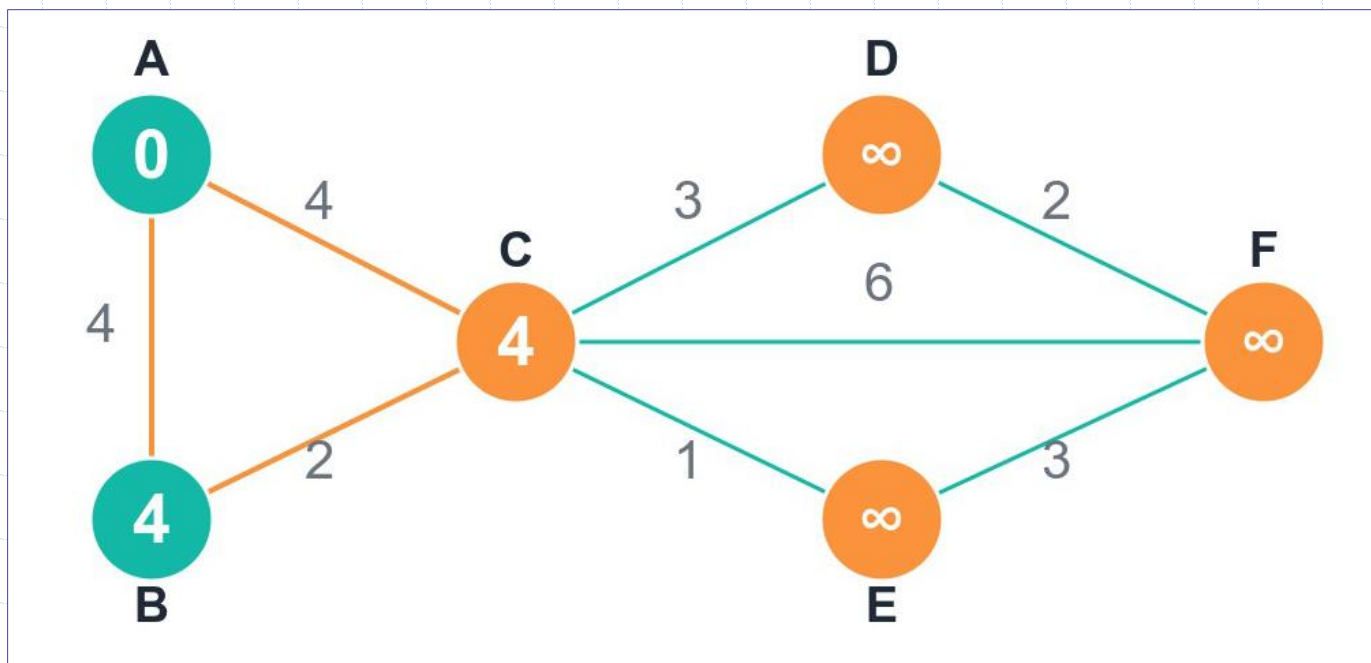
Dijkstra algoritam

Korak 2 – procesiranje čvora A: Počevši od čvora A, algoritam ispituje sve direktno povezane susede, a to su čvorovi B i C, svaki sa težinom 4. Rastojanja za ove čvorove se izračunavaju na sl. način: udaljenost do B ($0+4=4$), udaljenost do C ($0+4=4$). Pošto su obe udaljenosti manje od prethodno postavljenih vrednosti (∞), one se ažuriraju. Preostali čvorovi (D, E, F) zadržavaju svoje početne udaljenosti od ∞ .



Dijkstra algoritam

Korak 3 – procesiranje čvora B: Algoritam bira neposećeni čvor sa najmanjim rastojanjem. Čvorovi B i C imaju isto rastojanje 4, algoritam bira jedan od njih, npr. čvor B. Od čvora B, proveravaju se njegovi neposećeni susedi. B je povezan sa C granom težine 2, pa će rastojanje do C biti B ($4+2=6$) a to je veće od trenutne udaljenosti C (4), pa se ne vrši ažuriranje. Algoritam bira sledeći neposećeni čvor sa najmanjom udaljenošću, a to je čvor C.



Dijkstra algoritam

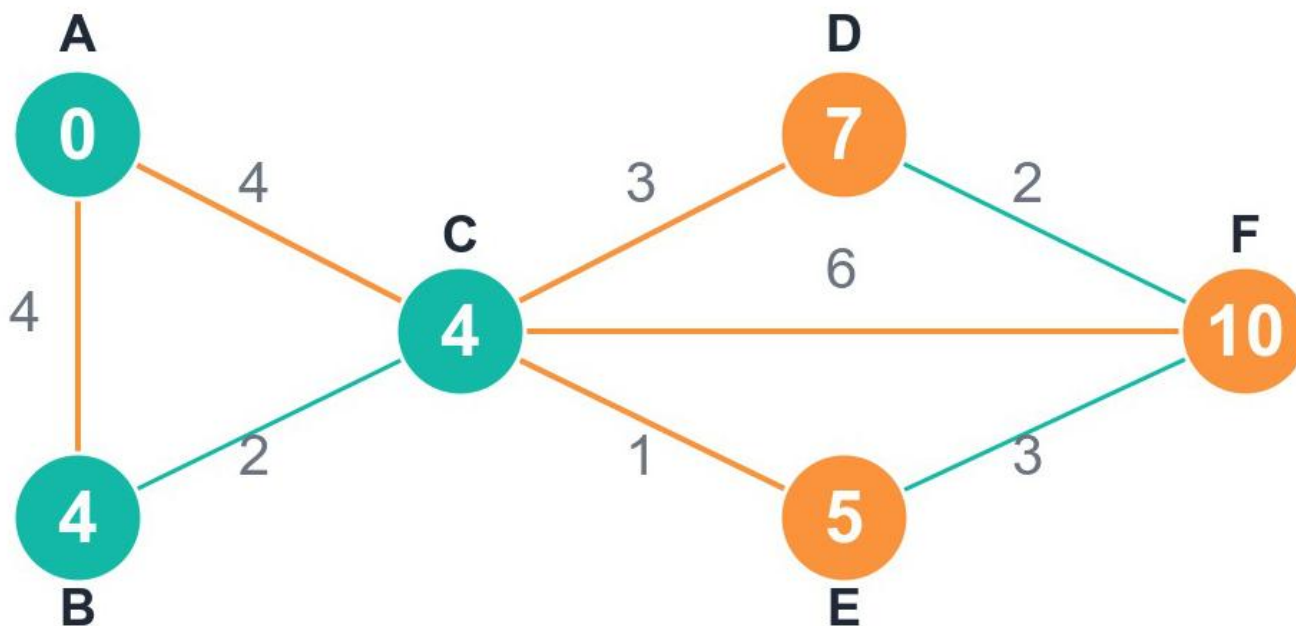
Korak 4 – procesiranje čvora C: Od čvora C (sa rastojanjem 4), ispituju se njegovi neposećeni susedi D, E i F. Raastojanja se izračunavaju na sledeći način:

Rastojanje do D: $4 + 3 = 7$

Rastojanje do E: $4 + 1 = 5$

Rastojanje do F: $4 + 6 = 10$

Sve ove vrednosti su manje od trenutnih rastojanja (∞), pa se vrednosti rstojanja čvorova ažuriraju

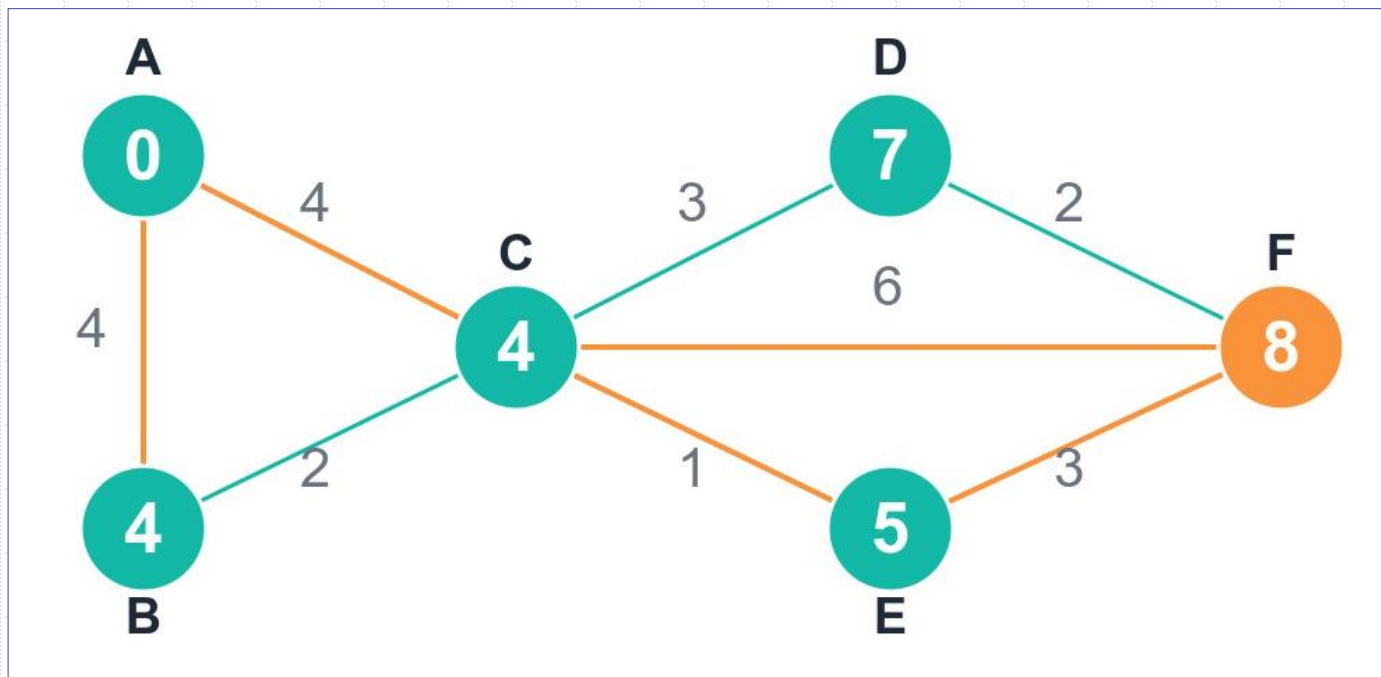


Dijkstra algoritam

Korak 5 – procesiranje čvora E: Sledeći neposećeni čvor sa najmanjim rastojanjem je E (rastojanje je 5). Iz čvora E algoritam proverava njegove neposećene susedne čvorove D i F:

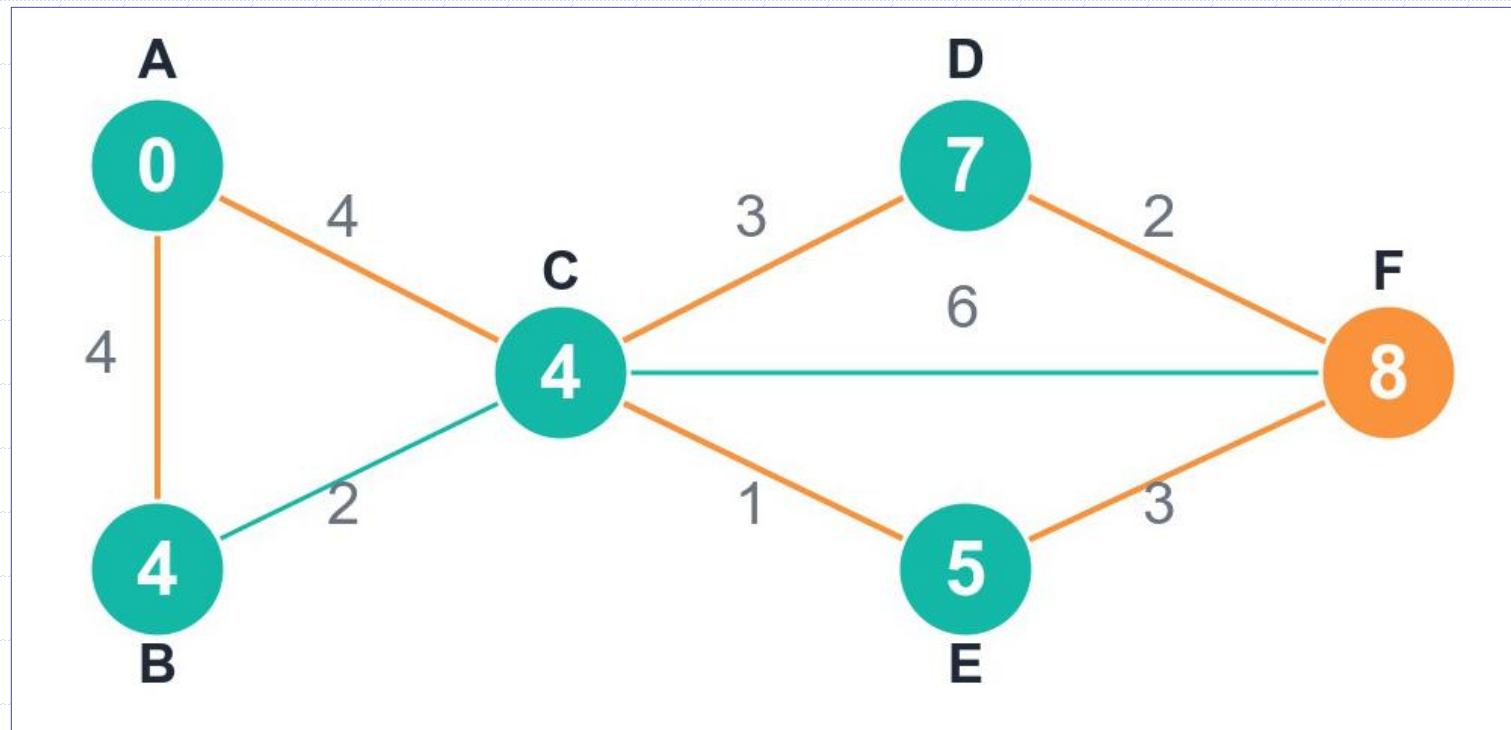
Udaljenost do D (preko E): $5 + 3 = 8 \rightarrow$ veća od trenutnog rastojanja D (7), tako da nema ažuriranja

Udaljenost do F (preko E): $5 + 3 = 8 \rightarrow$ manja od trenutnog rastojanja F (10), tako da se rastojanje na čvoru F ažurira na 8



Dijkstra algoritam

Korak 6 – procesiranje čvora D: Sledeći čvor sa najmanjim rastojanjem je D (rastojanje 7). Njegov jedini neposećeni susedni čvor je F, sa kojim je povezan granom težine 2. Rastojanje do F (preko D) se izračunava kao $7 + 2 = 9$, a pošto je veće 9 od trenutnog rastojanja čvora F (8), ne vrši se ažuriranje.

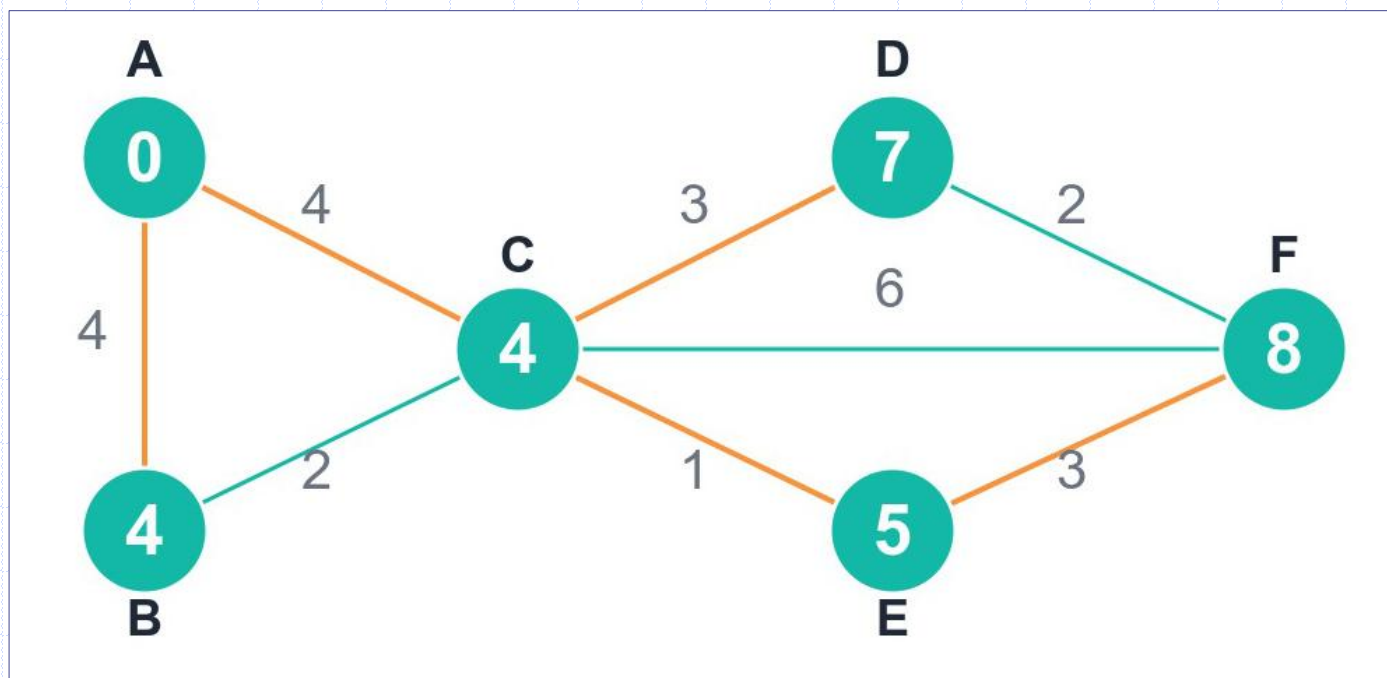


Dijkstra algoritam

Korak 6 – procesiranje čvora D: Nakon obilaska svih čvorova grafa, algoritam se završava. Konačna najkraća rastojanja od izvornog čvora (A) do svih ostalih čvorova su:

A = 0 B = 4 C = 4 D = 7 E = 5 F = 8

Ove rastojanja predstavljaju najkraću putanju od čvora A, sa minimalnim „troškovima“ za dostizanje svakog drugog čvora u grafu.



Složenost Dijkstra algoritma

- Vremenska složenost Dijkstra algoritma zavisi od strukture podataka koju koristimo za predstavljanje grafa i upravljanje prioritetnim redom
- Kada se u grafu koristi lista susedstva i red (hip) minimalnog prioriteta, vremenska složenost je **$O((V+E)\log V)$** gde **V** označava ukupan broj čvorova, a **E** označava ukupan broj grana grafa (svaki čvor se obrađuje jednom, a svaka grana može zahtevati ažuriranje prioritetnog reda)
- Kada se u grafu koristi matrica susedstva sa jednostavnom linearnom pretragom za najmanju udaljenost, složenost se povećava na **$O(V^2)$** što je manje efikasno za velike ili „sparse“ grafove
- Prostorna složenost Dijkstra algoritma je **$O(V+E)$** , jer se čuvaju format grafa, tabela udaljenosti i prioritetni red. Izračunava se vrednost rastojanja za svaki čvor, a prioritetni red pmože da sadrži do V čvorova u najgorem slučaju
- Mada je Dijkstra algoritam vremenski efikasan, on zahteva dovoljno memorije za procesiranje rastojanja čvorova i efikasno upravljanje posećenim čvorovima, posebno za velike mreže ili guste grafove

